
16 C Under Windows

- Which Windows...
- Integers
- The Use of typedef
- Pointers in the 32-bit World
 - Memory Management
 - Device Access
- DOS Programming Model
- Windows Programming Model
 - Event Driven Model
- Windows Programming, a Closer Look
- The First Windows Program
- Hungarian Notation
- Summary
- Exercise

So far we have learnt every single keyword, operator and instruction available in C. Thus we are through with the language elements that were there to learn. We did all this learning by compiling our programs using a 16-bit compiler like Turbo C/C++. Now it is time to move on to more serious stuff. To make a beginning one has to take a very important decision—should we attempt to build programs that are targeted towards 16-bit environments like MS-DOS or 32-bit environments like Windows/Linux. Obviously we should choose the 32-bit platform because that is what is in commercial use today and would remain so until 64-bit environment takes over in future. That raises a very important question—is it futile to learn C programming using 16-bit compiler like Turbo C/C++? Absolutely not! The typical 32-bit environment offers so many features that the beginner is likely to feel lost. Contrasted with this, 16-bit compilers offer a very simple environment that a novice can master quickly.

Now that the C fundamentals are out of the way and you are confident about the language features it is time for us to delve into the modern 32-bit operating environments. In today's commercial world 16-bit operating environments like DOS are more or less dead. More and more software is being created for 32-bit environments like Windows and Linux. In this chapter we would explore how C programming is done under Windows. Chapters 20 & 21 are devoted to exploring C under Linux.

Which Windows...

To a common user the differences amongst various versions of Windows like Windows 95, 98, ME, NT, 2000, XP, Server 2003 is limited to only visual appearances—things like color of the title bar, shape of the buttons, desktop, task bar, programs menu etc. But the truth is much farther than that. Architecturally there are huge differences amongst them. So many are the differences that Microsoft categorizes the different versions under two major heads—Consumer Windows and Windows NT Family. Windows

95, 98, ME fall under the Consumer Windows, whereas Windows NT, 2000, XP, Server 2003 fall under the Windows NT Family. Consumer Windows was targeted towards the home or small office users, whereas NT family was targeted towards business users. Microsoft no longer provides support for Consumer Windows. Hence in this book we would concentrate only on NT Family Windows. So in the rest of this book whenever I refer to Windows I mean Windows NT family, unless explicitly specified.

Before we start writing C programs under Windows let us first see some of the changes that have happened under Windows environment.

Integers

Under 16-bit environment the size of integer is of 2 bytes. As against this, under 32-bit environment an integer is of 4 bytes. Hence its range is -2147483648 to +2147483647. Thus there is no difference between an **int** and a **long int**. But what if we wish to store the age of a person in an integer? It would be improper to sacrifice a 4-byte integer when we know that the number to be stored in it is hardly going to exceed hundred. In such as case it would be more sensible to use a **short int** since it is only 2 bytes long.

The Use of *typedef*

Take a look at the following declarations:

```
COLORREF color ;  
HANDLE h ;  
WPARAM w ;  
LPARAM l ;  
BOOL b ;
```

Are COLORREF, HANDLE, etc. new datatypes that have been added in C under Windows compiler? Not at all. They are merely typedef's of the normal integer datatype.

A typical C under Windows program would contain several such **typedefs**. There are two reasons why Windows-based C programs heavily make use of **typedefs**. These are:

- (a) A typical Windows program is required to perform several complex tasks. For example a program may print documents, send mails, perform file I/O, manage multiple threads of execution, draw in a window, play sound files, perform operations over the network apart from normal data processing tasks. Naturally a program that carries out so many tasks would be very big in size. In such a program if we start using the normal integer data type to represent variables that hold different entities we would soon lose track of what that integer value actually represents. This can be overcome by suitably typedefing the integer as shown above.
- (b) At several places in Windows programming we are required to gather and work with dissimilar but inter-related data. This can be done using a structure. But when we define any structure variable we are required to precede it with the keyword **struct**. This can be avoided by using **typedef** as shown below:

```
struct rect
{
    int top ;
    int left ;
    int right ;
    int bottom ;
};

typedef struct rect RECT ;
typedef struct rect* PRECT ;
```

```
RECT r ;  
PRECT pr ;
```

What have we achieved out of this? It makes user-defined data types like structures look, act and behave similar to standard data types like integers, floats, etc. You would agree that the following declarations

```
RECT r ;  
int i ;
```

are more logical than

```
struct RECT r ;  
int i ;
```

Imagine a situation where each programmer **typedefs** the integer to represent a color in different ways. Some of these could be as follows:

```
typedef int COL ;  
typedef int COLOR ;  
typedef int COLOUR ;  
typedef int COLORREF ;
```

To avoid this chaos Microsoft has done several **typedefs** for commonly required entities in Windows programming. All these have been stored in header files. These header files are provided as part of 32-bit compiler like Visual C++.

Pointers in the 32-bit World

In a 16-bit world (like MS-DOS) we could run only one application at a time. If we were to run another program we were required to terminate the first one before launching the second. As only one program (task) could run at a time this environment was

called single-tasking environment. Since only one program could run at any given time entire resources of the machine like memory and hardware devices were accessible to this program. Under 32-bit environment like Windows several programs reside and work in memory at the same time. Hence it is known as a multi-tasking environment. But the moment there are multiple programs running in memory there is a possibility of conflict if two programs simultaneously access the machine resources. To prevent this, Windows does not permit any application direct access to any machine resource. To channelize the access without resulting into conflict between applications several new mechanisms were created in the Microprocessor & OS. This had a direct bearing on the way the application programs are created. This is not a Windows OS book. So we would restrict our discussion about the new mechanisms that have been introduced in Windows to topics that are related, to C programming. These topics are ‘Memory Management and Device Access’.

Memory Management

Since users have become more demanding, modern day applications have to contend with these demands and provide several features in them. To add to this, under Windows several such applications run in memory simultaneously. The maximum allowable memory—1 MB—that was used in 16-bit environment was just too small for this. Hence Windows had to evolve a new memory management model. Since Windows runs on 32-bit microprocessors each CPU register is 32-bit long. Whenever we store a value at a memory location the address of this memory location has to be stored in the CPU register at some point in time. Thus a 32-bit address can be stored in these registers. This means that we can store 2^{32} unique addresses in the registers at different times. As a result, we can access 4 GB of memory locations using 32-bit registers. As pointers store addresses, every pointer under 32-bit environment also became a 4-byte entity.

However, if we decide to install 4 GB memory it would cost a lot. Hence Windows uses a memory model which makes use of as much of physical memory (say 128 MB) as has been installed and simulates the balance amount of memory (4 GB – 128 MB) on the hard disk. Be aware that this balance memory is simulated as and when the need to do so arises. Thus memory management is demand based.

Note that programs cannot execute straight-away from hard disk. They have to be first brought into physical memory before they can get executed. Suppose there are multiple programs already in memory and a new program starts executing. If this new program needs more memory than what is available right now, then some of the existing programs (or their parts) would be transferred to the disk in order to free the physical memory to accommodate the new program. This operation is often called page-out operation. Here page stands for a block of memory (usually of size 4096 bytes). When that part of the program that was paged out is needed it is brought back into memory (called page-in operation) and some other programs (or their parts) are paged out. This keeps on happening without a common user's knowledge all the time while working with Windows. A few more facts that you must note about paging are as follows:

- (a) Part of the program that is currently executing might also be paged out to the disk.
- (b) When the program is paged in (from disk to memory) there is no guarantee that it would be brought back to the same physical location where it was before it was paged out.

Now imagine how the paging operations would affect our programming. Suppose we have a pointer pointing to some data present in a page. If this page gets paged out and is later paged in to a different physical location then the pointer would obviously have a wrong address. Hence under Windows the pointer never holds the physical address of any memory location. It always holds a virtual address of that location. What is this virtual address? At

its name suggests it is certainly not a real address. It is a number, which contains three parts. These parts when used in conjunction with a CPU register called CR3 and contents of two tables called Page Directory Table and Page Table leads to the actual physical address. This is shown in Figure 16.1.

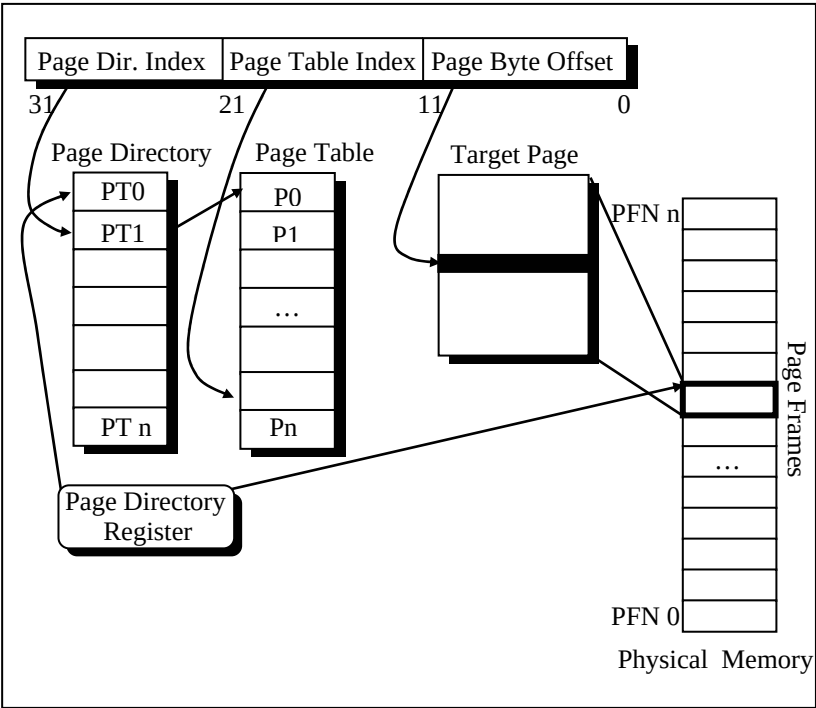


Figure 16.1

The CR3 register holds the physical location of Page Directory Table. The left part of the 32-bit virtual address holds the index into the Page Directory Table. The value present at this index is the starting address of the Page Table. The middle part of the 32-bit virtual address holds the index into the Page Table. The value present at this index is the starting address of the physical page in memory. The right part of the 32-bit virtual address holds the byte

offset (from the start of the page) of the physical memory location to be accessed.

Note that the CR3 register is not accessible from an application. Hence an application can never directly reach a physical address. Also, as the paging activity is going on the OS would suitably keep updating the values in the two tables.

Device Access

All devices under Windows are shared amongst all the running programs. Hence no program is permitted a direct access to any of the devices. The access to a device is routed through a device driver program, which finally accesses the device. There is a standard way in which an application can communicate with the device driver. It is device driver's responsibility to ensure that multiple requests coming from different applications are handled without causing any conflict. This standard way of communication is discussed in detail in Chapter 17.

DOS Programming Model

Typical 16-bit environments like DOS use a sequential programming model. In this model programs are executed from top to bottom in an orderly fashion. The path along which the control flows from start to finish may vary during each execution depending on the input that the program receives or the conditions under which it is run. However, the path remains fairly predictable. C programs written in this model begin execution with **main()** (often called entry point) and then call other functions present in the program. If you assume some input data you can easily walk through the program from beginning to end. In this programming model it is the program and not the operating system that determines which function gets called and when. The operating system simply loads and executes the program and then waits for it to finish. If the program wishes it can take help of the OS to carry

out jobs like console I/O, file I/O, printing, etc. For other operations like generating graphics, carrying out serial communication, etc. the program has to call another set of functions called ROM-BIOS functions.

Unfortunately the DOS functions and the BIOS functions do not have any names. Hence to call them the program had to use a mechanism called interrupts. This is a messy affair since the programmer has to remember interrupt numbers for calling different functions. Moreover, communication with these functions has to be done using CPU registers. This lead to lot of difficulties since different functions use different registers for communication. To an extent these difficulties are reduced by providing library functions that in turn call the DOS/BIOS functions using interrupts. But the library doesn't have a parallel function for every DOS/BIOS function. DOS functions either call BIOS functions or directly access the hardware.

At times the programs are needed to directly interact with the hardware. This has to be done because either there are no DOS/BIOS functions to do this, or if they are there their reach is limited.

Figure 16.2 captures the essence of the DOS programming model.

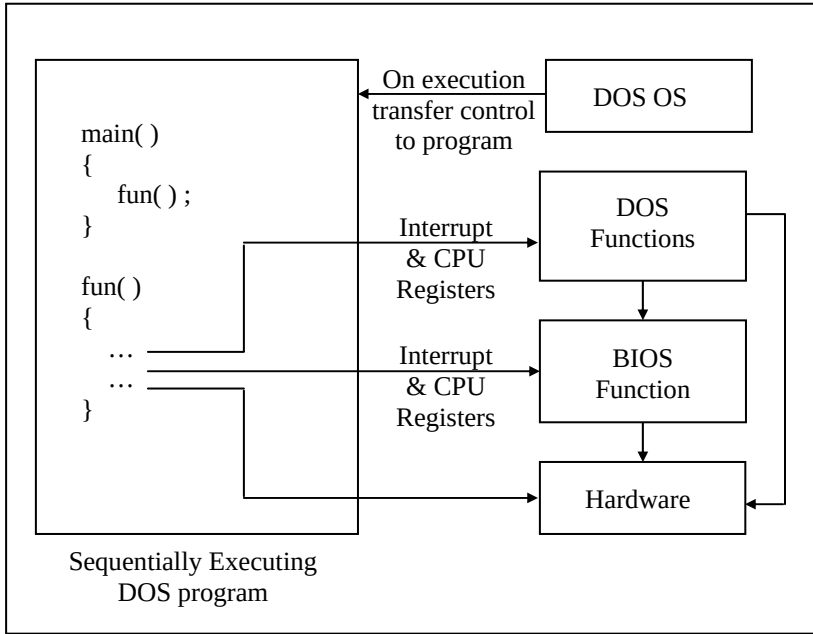


Figure 16.2

From the above discussion you can gather that there are several limitations in the DOS programming model. These have been listed below:

No True Reuse

The library functions that are called from each program become part of the executable file (.EXE) for that program. Thus the same functions get replicated in several EXE files, thereby wasting precious disk space.

Inconsistent Look and Feel

Every DOS program has a different user interface that the user has to get used to before he can start getting work out of the program. For example, successful DOS-based software like Lotus 1-2-3, Foxpro, Wordstar offered different types of menus. This happened because DOS/BIOS doesn't provide any functions for creating user interface elements like menus. As the look and feel of all DOS based programs is different, the user takes a lot of time in learning how to interact with the program

Messy Calling Mechanism

It is difficult to remember interrupt numbers and the registers that are to be used for communication with DOS/BIOS functions. For example, if we are to position the cursor on the screen using a BIOS function we are required to remember the following details:

Interrupt number – 16

CPU Registers to be used:

AH – 2 (service number)

DH – Row number where cursor is to be positioned

DL – Column number where cursor is to be positioned

While using these interrupt numbers and registers there is always a chance of error.

Hardware Dependency

DOS programs are always required to bother about the details of the hardware on which they are running. This is because for every new piece of hardware introduced there are new interrupt numbers and new register details. Hence DOS programmers are under the constant fear that if the hardware on which the programs are running changes then the program may crash.

Moreover the DOS programmer has to write lot of code to detect the hardware on which his program is running and suitably make use of the relevant interrupts and registers. Not only does this make the program lengthy, the programmer has to understand a lot of technical details of the hardware. As a result the programmer has to spend more time in understanding the hardware than in the actual application programming.

Windows Programming Model

From the perspective of the user the shift from MS-DOS to Windows OS involves switching over to a Graphical User Interface from the typical Text Interface that MS-DOS offers. Another change that the user may feel and appreciate is the ability of Windows OS to execute several programs simultaneously, switching effortlessly from one to another by pointing at windows and clicking them with the mouse. Mastering this new GUI environment and getting comfortable with the multitasking feature is at the most a matter of a week or so. However, from the programmer's point of view programming for Windows is a whole new ball game!

Windows programming model is designed with a view to:

- (a) Eliminate the messy calling mechanism of DOS
- (b) Permit true reuse of commonly used functions
- (c) Provide consistent look and feel for all applications
- (d) Eliminate hardware dependency

Let us discuss how Windows programming model achieves this.

Better Calling Mechanism

Instead of calling functions using Interrupt numbers and registers Windows provides functions within itself which can be called using names. These functions are called API (Application Programming Interface) functions. There are literally hundreds of

API functions available. They help an application to perform various tasks such as creating a window, drawing a line, performing file input/output, etc.

True Reuse

A C under Windows program calls several API functions during course of its execution. Imagine how much disk space would have been wasted had each of these functions become part of the EXE file of each program. To avoid this, the API functions are stored in special files that have an extension .DLL.

DLL stands for Dynamic Link Libraries. A DLL is a binary file that provides a library of functions. The functions present in DLLs can be linked during execution. These functions can also be shared between several applications running in Windows. Since linking is done dynamically the functions do not become part of the executable file. As a result, the size of EXE files does not go out of hand. It is also possible to create your own DLLs. You would like to do this for two reasons:

- (a) Sharing common code between different executable files.
- (b) Breaking an application into component parts to provide a way to easily upgrade application's components.

The Windows API functions come in three DLL files. Figure 16.3 lists these filenames along with purpose of each.

DLL	Description
USER32.DLL	Contains functions that are responsible for window management, including menus, cursors, communications, timer etc.
GDI32.DLL	Contains functions for graphics drawing and painting
KERNEL32.DLL	Contains functions to handle memory management, threading, etc.

Figure 16.3

Consistent Look and Feel

Consistent look and feel means that each program offers a consistent and similar user interface. As a result, user doesn't have to spend long periods of time mastering a new program. Every program occupies a window—a rectangular area on the screen. A window is identified by a title bar. Most program functions are initiated through the program's menu. The display of information too large to fit on a single screen can be viewed using scroll bars. Some menu items invoke dialog boxes, into which the user enters additional information. One dialog box is found in almost every Windows program. It opens a file. This dialog box looks the same (or very similar) in many different Windows programs, and it is almost always invoked from the same menu option.

Once you know how to use one Windows program, you're in a good position to easily learn another. The menus and dialog boxes allow user to experiment with a new program and explore its features. Most Windows programs have both a keyboard interface and a mouse interface. Although most functions of Windows programs can be controlled through the keyboard, using the mouse is often easier for many chores.

From the programmer's perspective, the consistent user interface results from using the Windows API functions for constructing menus and dialog boxes. All menus have the same keyboard and mouse interfaces because Windows—rather than the application program—handles this job.

Hardware Independent Programming

As we saw earlier a Windows program can always call Windows API functions. Thus an application can easily communicate with OS. What is new in Windows is that the OS can also communicate with application. Let us understand why it does so with the help of an example.

Suppose we have written a program that contains a menu item, which on selection is supposed to display a string “Hello World” in the window. The menu item can be selected either using the keyboard or using the mouse. On executing this program it will perform the initializations and then wait for the user input. Sooner or later the user would press the key or click the mouse to select the menu-item. This key-press or mouse-click is known as an ‘event’. The occurrence of this event is sensed by the keyboard or mouse device driver. The device driver would now inform Windows about it. Windows would in turn notify the application about the occurrence of this event. This notification is known as a ‘message’. Thus the OS has communicated with the application. When the application receives the message it communicates back with the OS by calling a Windows API function to display the string “Hello World” in the window. This API function in turn communicates with the device driver of the graphics card (that drives the screen) to display the string. Thus there is a two-way communication between the OS and the application. This is shown in Figure 16.4.

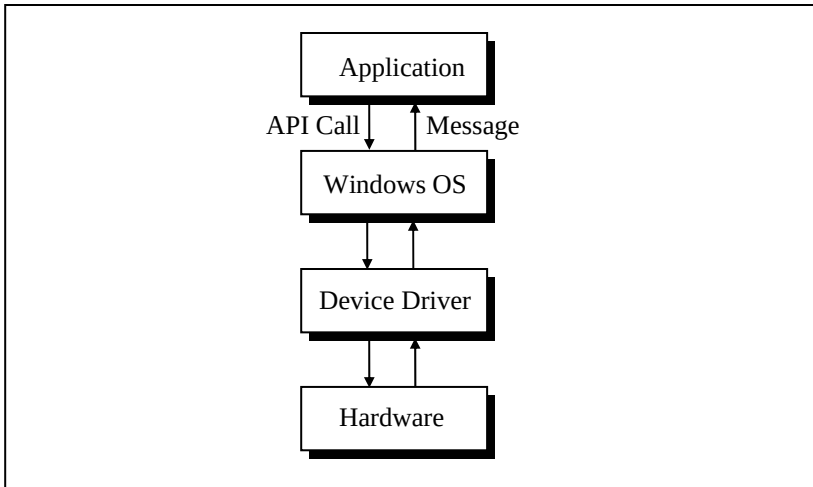


Figure 16.4

Suppose the keyboard and the mouse are now replaced with a new keyboard and mouse. Doing so would not affect the application at all. This is because at no time does the application carry out any direct communication with the devices. Any differences that may be there in the new set of mouse and keyboard would be handled the device driver and not by the application program. Similarly, if the screen or the graphics card is replaced no change would be required in the program. In short hardware independence at work! At times a change of device may necessitate a change in the device driver program, but never a change in the application.

Event Driven Model

When a user interacts with a Windows program a lot of events occur. For each event a message is sent to the program and the program reacts to it. Since the order in which the user would interact with the user-interface elements of the program cannot be predicted the order of occurrence of events, and hence the order of messages, also becomes unpredictable. As a result, the order of

calling the functions in the program (that react to different messages) is dictated by the order of occurrence of events. Hence this programming model is called ‘Event Driven Programming Model’.

That’s really all that is there to event-driven programming. Your job is to anticipate what users are likely to do with your application’s user interface objects and have a function waiting, ready to execute at the appropriate time. Just when that time is, no one except the user can really say.

Windows Programming, a Closer Look

There can be hundreds of ways in which the user may interact with an application. In addition to this some events may occur without any user interaction. For example, events occur when we create a window, when the window’s contents are to be drawn, etc. Not only this, occurrence of one event may trigger a few more events. Thus literally hundreds of messages may be sent to an application thereby creating a chaos. Naturally, a question comes—in which order would these messages get processed by the application. Order is brought to this chaos by putting all the messages that reach the application into a ‘Queue’. The messages in the queue are processed in First In First Out (FIFO) order.

In fact the OS maintains several such queues. There is one queue, which is common for all applications. This queue is known as ‘System Message Queue’. In addition there is one queue per application. Such queues are called ‘Application Message Queues’. Let us understand the need for maintaining so many queues.

When we click a mouse and an event occurs the device driver posts a message into the System Message Queue. The OS retrieves this message finds out with regard to which application the message has been sent. Next it posts a message into the

Application Message Queue of the application in which the mouse was clicked. Refer Figure 16.5.

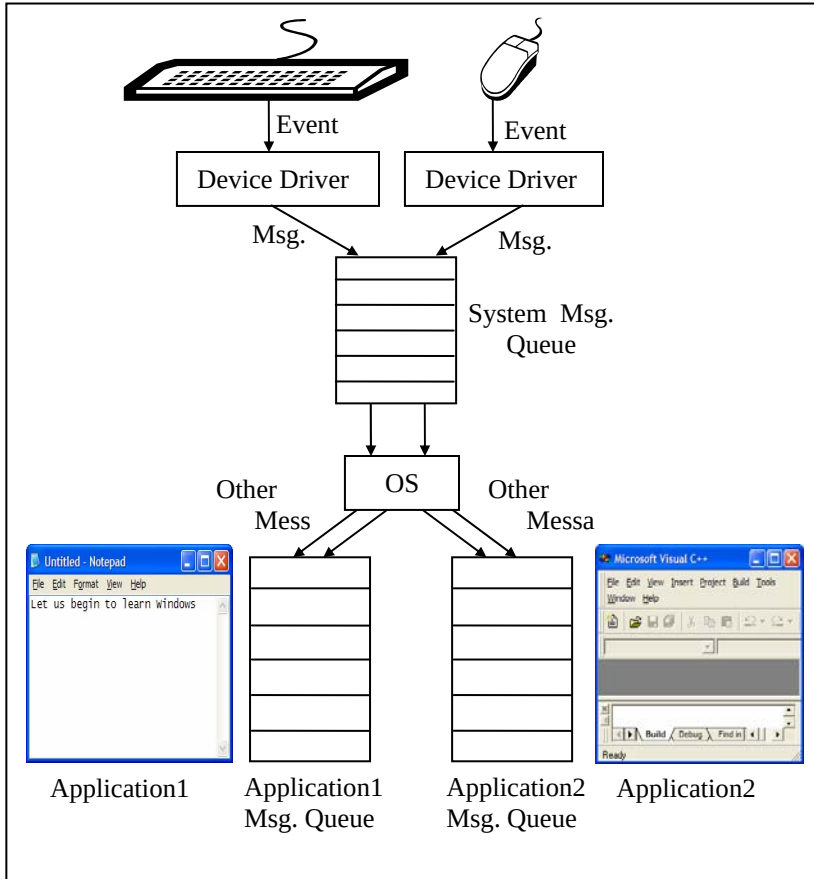


Figure 16.5

I think now we have covered enough ground to be able to actually start C under Windows programming. Here we go...

The First Windows Program

To keep things simple we would begin with a program that merely displays a “Hello” message in a message box. Here is the program...

```
#include <windows.h>
int _stdcall WinMain ( HINSTANCE hInstance, HINSTANCE hPrevInstance,
                      LPSTR lpszCmdline, int nCmdShow )
{
    MessageBox ( 0, "Hello!", "Title", 0 );
    return ( 0 );
}
```

Naturally a question would come to your mind—how do I create and run this program and what output does it produce. Firstly take a look at the output that it produces. Here it is...



Figure 16.6

Let us now look at the steps that one needs to carry to create and execute this program:

- Start VC++ from ‘Start | Programs | Microsoft Visual C++ 6.0’. The VC++ IDE window will get displayed.
- From the File | New menu, select ‘Win32 Application’, and give a project name, say, ‘sample1’. Click on OK.
- From the File | New menu, select ‘C++ Source File’, and give a suitable file name, say, ‘sample1’. Click on OK.
- The ‘Win32 Application-Step 1 of 1’ window will appear. Select ‘An empty project’ option and click ‘Finish’ button.

- (e) A 'New Project Information' dialog will appear. Close it by clicking on OK.
- (f) Again select 'File | New | C++ Source File'. Give the file name as 'sample1.c'. Click on OK.
- (g) Type the program in the 'sample1.c' file that gets opened in the VC++ IDE.
- (h) Save this file using 'Save' option from the File menu.

To execute the program follow the steps mentioned below:

- (a) From the Build menu, select 'Build sample1.exe'.
- (b) Assuming that no errors were reported in the program, select 'Execute sample1.exe' from the Build menu.

Let us now try to understand the program. The way every C under DOS program begins its execution with **main()**, every C under Windows program begins its execution with **WinMain()**. Thus **WinMain()** becomes the entry point for a Windows program. A typical **WinMain()** looks like this:

```
int __stdcall WinMain ( HINSTANCE hInstance, HINSTANCE hPrevInstance,  
                      LPSTR lpszCmdLine, int nCmdShow )
```

Note the **__stdcall** before **WinMain()**. It indicates the calling convention used by the **WinMain()** function. Calling Conventions indicate two things:

- (a) The order (left to right or right to left) in which the arguments are pushed onto the stack when a function call is made.
- (b) Whether the caller function or called function removes the arguments from the stack at the end of the call.

Out of the different calling conventions available most commonly used conventions are **__cdecl** and **__stdcall**. Both these calling conventions pass arguments to functions from right to left. In **__cdecl** the stack is cleaned up by the calling function, whereas in case of **__stdcall** the stack is cleaned up by the called function. All

API functions use **__stdcall** calling convention. If not mentioned, **__cdecl** calling convention is assumed by the compiler.

HINSTANCE and LPSTR are nothing but **typedefs**. The first is an **unsigned int** and the second is a **pointer** to a **char**. These macros are defined in 'windows.h'. This header file must always be included while writing a C program under Windows. **hInstance**, **hPrevInstance**, **lpszCmdLine** and **nCmdShow** are simple variable names. In place of these we can use **i**, **j**, **k** and **l** respectively. Let us now understand the meaning of these parameters as well as the rest of the program.

- **WinMain()** receives four parameters which are as under:

hInstance: This is the 'instance handle' for the running application. Windows creates this ID number when the application starts. We will use this value in many Windows functions to identify an application's data.

A handle is simply a 32-bit number that refers to an entity. The entity could be an application, a window, an icon, a brush, a cursor, a bitmap, a file, a device or any such entity. The actual value of the handle is unimportant to your programs, but the Windows module that gives your program the handle knows how to use it to refer to an entity. What is important is that there is a unique handle for each entity and we can refer and reach the entity only using its handle.

hPrevInstance: This parameter is a remnant of earlier versions of Windows and is no longer significant. Now it always contains a value 0. It is being persisted with only to ensure backward compatibility.

lpszCmdLine: This is a pointer to a character string containing the command line arguments passed to the program. This is similar to the **argv**, **argc** parameters passed to **main()** in a DOS program.

nCmdShow: This is an integer value that is passed to the function. This integer tells the program whether the window that it creates should appear minimized, as an icon, normal, or maximized when it is displayed for the first time.

- The **MessageBox()** function pops up a message box whose title is ‘Title’ and which contains a message ‘Hello!’.
- Returning **0** from **WinMain()** indicates success, whereas, returning a nonzero value indicates failure.
- Instead of printing ‘Hello!’ in the message box we can print the command line arguments that the user may supply while executing the program. The command line arguments can be supplied to the program by executing it from Start | Run as shown in Figure 16.7.

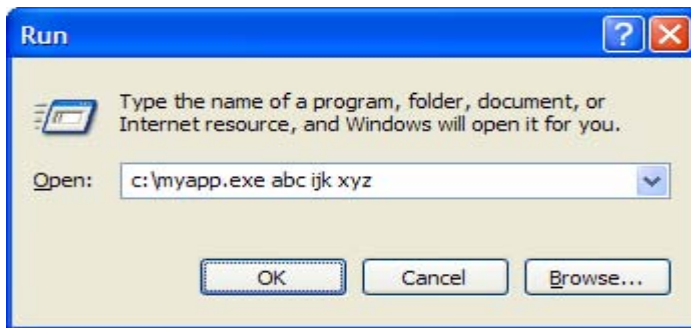


Figure 16.7

Note from Figure 16.7 that ‘myapp.exe’ is the name of our application, whereas, ‘abc ijk xyz’ represents command line arguments. The parameter **lpszCmdline** points to the string “abc ijk xyz”. This string can be printed using the following statement:

```
MessageBox ( 0, lpszCmdline, "Title", 0 ) ;
```

If the entire command line including the filename is to be retrieved we can use the **GetCommandLine()** function.

Hungarian Notation

Hungarian Notation is a variable-naming convention so called in the honor of the legendary Microsoft programmer Charles Simonyi. According to this convention the variable name begins with a lower case letter or letters that denotes the data type of the variable. For example, the **sz** prefix in **szCmdLine** stands for 'string terminated by zero'; the prefix **h** in **hInstance** stands for 'handle'; the prefix **n** in **nCmdShow** stands for **int**. Prefixes are often combined to form other prefixes, as **lpsz** in **lpszCmdLine** stands for 'long pointer to a zero terminated string'. Though basically this notation is a good idea nowadays its usage is discouraged. This is because when a transition happens from say a 16-bit code to 32-bit code then a whole lot of variable names have to be changed. For example, suppose the 16-bit code used 2-byte and 4-byte integer variables called **wParam** and **lParam**, where **w** indicated a 16-bit integer (word) and a 32-bit integer (long) respectively. When this code is ported to a 32-bit environment **wParam** had to be changed to **lParam** since in this environment every integer is 4 bytes long. You would agree that if we follow the Hungarian notation then we would have to make a whole lot of changes in the variable names when we port the code to a 32-bit or a 64-bit environment. Hence the usage of this convention is nowadays discouraged.

Summary

- (a) Under Windows an integer is four bytes long. To use a two-byte integer pre-qualify it with **short**.
- (b) Under Windows a pointer is four bytes long.
- (c) Windows programming involves a heavy usage of **typedefs**.
- (d) DOS uses a Sequential Programming Model, whereas, Windows uses an Event Driven Programming Model.
- (e) Entry point of every Windows program is a function called **WinMain()**.

- (f) Windows does not permit direct access to memory or hardware devices.
- (g) Windows uses a Demand-based Virtual Memory Model to manage memory.
- (h) Under Windows there is two-way communication between the program and the OS.
- (i) Windows maintains a system message queue common for all applications.
- (j) Windows maintains an application message queue per running application.
- (k) Calling convention decides the order in which the parameters are passed to a function and whether the calling function or the called function clears the stack.
- (l) Commonly used calling conventions are **__cdecl** and **__stdcall**.
- (m) Hungarian notation though good its usage is not recommended any more.

Exercise

[A] State True or False:

- (a) MS-DOS uses a procedural programming model.
- (b) A Windows program can directly call a device driver program for a device.
- (c) API functions under Windows do not have names.
- (d) DOS functions are called using an interrupt mechanism.
- (e) Windows uses a 4 GB virtual memory space.
- (f) Size of a pointer under Windows depends upon whether it is **near** or **far**.
- (g) Under Windows the address stored in a pointer is a virtual address and not a physical address.
- (h) One of the parameters of **WinMain()** called **hPrevInstance** is no longer relevant.

[B] Answer the following:

- (a) Why is Event-driven Programming Model better than the Sequential Programming Model?
- (b) What is the meaning of different parts of the address stored in a pointer under Windows environment?
- (c) Why Windows does not permit direct access to hardware?
- (d) What is the difference between an event and a message?
- (e) Why Windows maintains a different message queue for each application?
- (f) In which different situations messages get posted into an application message queue?

[C] Attempt the following:

- (a) Write a program that prints the value of **hInstance** in a message box.
- (b) Write a program that displays three buttons 'Yes', 'No' 'Cancel' in the message box.
- (c) Write a program that receives a number as a command line argument and prints its factorial value in a message box.
- (d) Write a program that displays command line arguments including file name in a message box.

17 *Windows Programming*

- The Role of a Message Box
- Here comes the window...
- More Windows
- A Real-World Window
 - Creation and Displaying of Window
 - Interaction with Window
 - Reacting to Messages
- Program Instances
- Summary
- Exercise

Event driven programming requires a change in mind set. I hope Chapter 16 has been able to bring about this change. However this change would be bolstered by writing event driven programs. This is what this chapter intends to do. I am hopeful that by the time you reach the end of this chapter you would be so comfortable with it as if you have been using it all your life.

The Role of a Message Box

Often we are required to display certain results on the screen during the course of execution of a program. We do this to ascertain whether we are getting the results as per our expectations. In a sequential DOS based program we can easily achieve this using **printf()** statements. Under Windows screen is a shared resource. So you can imagine what chaos would it create if all running applications are permitted to write to the screen. You would not be able to make out which output is of what application. Hence no Windows program is permitted to write anything directly to the screen. That's where a message box enters the scene. Using it we can display intermediate results during the course of execution of a program. It can be dismissed either by clicking the 'close button' in its title bar or by clicking the OK button present in it. There are numerous variations that you can try with the **MessageBox()**. Some of these are given below

```
MessageBox ( 0, "Are you sure", "Caption", MB_YESNO ) ;  
MessageBox ( 0, "Print to the Printer", "Caption", MB_YESNO CANCEL ) ;  
MessageBox ( 0, "icon is all about style", "Caption", MB_OK |  
            MB_ICONINFORMATION ) ;
```

You can put the above statements within **WinMain()** and see the results for yourself. Though the above message boxes give you flexibility in displaying results, button, icons, there is a limit to which you can stretch them. What if we want to draw a free hand drawing or display an image, etc. in the message box. This would

not be possible. To achieve this we need to create a full-fledged window. The next section discusses how this can be done.

Here Comes the window...

Before we proceed with the actual creation of a window it would be a good idea to identify the various elements of it. These are shown in Figure 17.1.

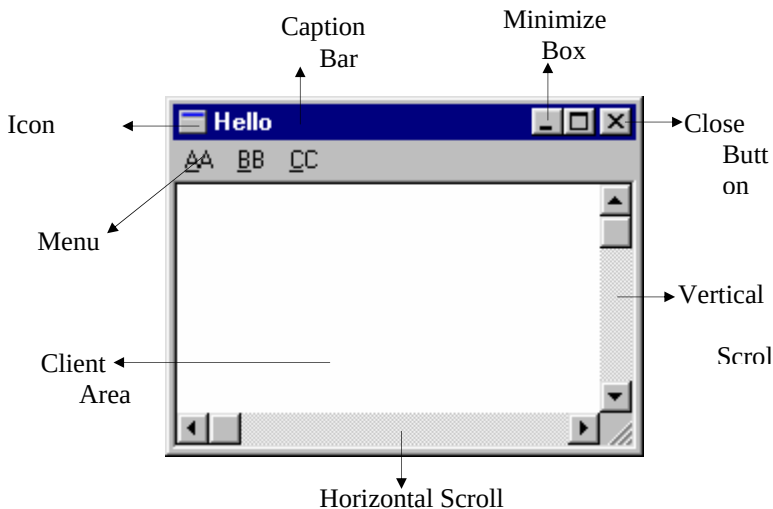


Figure 17.1

Note that every window drawn on the screen need not necessarily have every element shown in the above figure. For example, a window may not contain the minimize box, the maximize box, the scroll bars and the menu.

Let us now create a simple program that creates a window on the screen. Here is the program...

```
#include <windows.h>
```

```
int _stdcall WinMain ( HINSTANCE hInstance, HINSTANCE hPrevInstance,  
                      LPSTR lpszCmdLine, int nCmdShow )  
{  
    HWND h ;  
  
    h = CreateWindow ( "BUTTON", "Hit Me", WS_OVERLAPPEDWINDOW,  
                      10, 10, 150, 100, 0, 0, i, 0 ) ;  
    ShowWindow ( h, nCmdShow ) ;  
    MessageBox ( 0, "Hi!", "Waiting", MB_OK ) ;  
    return 0 ;  
}
```

Here is the output of the program...



Figure 17.2

Let us now understand the program. Every window enjoys certain properties—background color, shape of cursor, shape of icon, etc. All these properties taken together are known as ‘window class’. The meaning of ‘class’ here is ‘type’. Windows insists that a window class should be registered with it before we attempt to create windows of that type. Once a window class is registered we can create several windows of that type. Each of these windows would enjoy the same properties that have been registered through the window class. There are several predefined window classes. Some of these are `BUTTON`, `EDIT`, `LISTBOX`, etc. Our program has created one such window using the predefined `BUTTON` class.

To actually create a window we need to call the API function **CreateWindow()**. This function requires several parameters starting with the window class. The second parameter indicates the text that is going to appear on the button surface. The third parameter specifies the window style. **WS_OVERLAPPEDWINDOW** is a commonly used style. The next four parameters specify the window's initial position and size—the x and y screen coordinates of the window's top left corner and the window's width and height in pixels. The next three parameters specify the handles to the parent window, the menu and the application instance respectively. The last parameter is the pointer to the window-creation data.

We can easily devote a section of this book to **CreateWindow()** and its parameters. But don't get scared of it. Nobody is supposed to remember all the parameters, their meaning and their order. You can always use MSDN (Microsoft Developer Network) help to understand the minute details of each parameter. This help is available as part of VC++ 6.0 product. It is also available on the net at <http://www.msdn.microsoft.com/library>.

Note that **CreateWindow()** merely creates the window in memory. We still are to display it on the screen. This can be done using the **ShowWindow()** API function. **CreateWindow()** returns handle of the created window. Our program uses this handle to refer to the window while calling **ShowWindow()**. The second parameter passed to **ShowWindow()** signifies whether the window would appear minimized, maximized or normal. If the value of this parameter is **SW_SHOWNORMAL** we get a normal sized window, if it is **SW_SHOWMINIMIZED** we get a minimized window and if it is **SW_SHOWMAXIMIZED** we get a maximized window. We have passed **nCmdShow** as the second parameter. This variable contains **SW_SHOWNORMAL** by default. Hence our program displays a normal sized window.

The `WS_OVERLAPPEDWINDOW` style is a collection of the following styles:

```
WS_OVERLAPPED | WS_CAPTION | WS_SYSMENU | WS_THICKFRAME |  
WS_MINIMIZEBOX | WS_MAXIMIZEBOX
```

As you can make out from these macros they essentially control the look and feel of the window being created. All these macros are **#defined** in the 'Windows.h' header file.

On executing this program a window and a message box appears on the screen as shown in the Figure 17.2. The window and the message box disappear as soon as we click on OK. This is because on doing so execution of **WinMain()** comes to an end and moreover we have made no provision to interact with the window.

You can try to remove the call to **MessageBox()** and see the result. You would observe that no sooner does the window appear it disappears. Thus a call to **MessageBox()** serves the similar purpose as **getch()** does in sequential programming.

More Windows

Now that we know how to create a window let us create several windows on the screen. The program to do this is given below.

```
#include <windows.h>  
  
int _stdcall WinMain ( HINSTANCE hInstance, HINSTANCE hPrevInstance,  
                      LPSTR lpszCmdLine, int nCmdShow )  
{  
    HWND h[10];  
    int x;  
  
    for ( x = 0 ; x <= 9 ; x++ )  
    {
```



```
h[x] = CreateWindow ( "BUTTON", "Press Me",  
                    WS_OVERLAPPEDWINDOW, x * 20,  
                    x * 20, 150, 100, 0, 0, i, 0 );  
ShowWindow ( h[x], 1 );  
}  
  
MessageBox ( 0, "Hi!", "Waiting", 0 );  
return 0 ;  
}
```

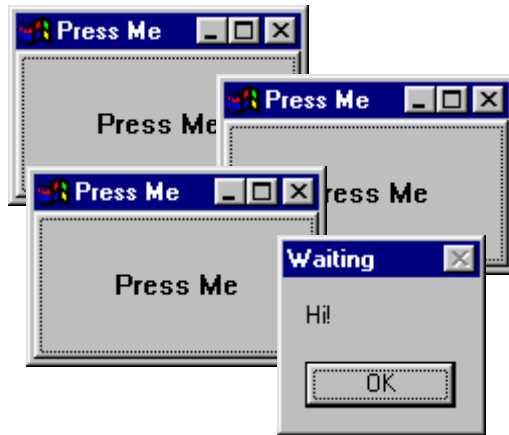


Figure 17.3

Note that each window created in this program is assigned a different handle. You may experiment a bit by changing the name of the window class to EDIT and see the result.

A Real-World Window

Suppose we wish to create a window and draw a few shapes in it. For creating such a window there is no standard window class available. Hence we would have to create our own window class, register it with Windows OS and then create a window on the basis of it. Instead of straightway jumping to a program that draws

shapes in a window let us first write a program that creates a window using our window class and lets us interact with it. Here is the program...

```
#include <windows.h>
#include "helper.h"

void OnDestroy ( HWND ) ;

int __stdcall WinMain ( HINSTANCE hInstance, HINSTANCE hPrevInstance,
                        LPSTR lpszCmdline, int nCmdShow )
{
    MSG m ;

    /* perform application initialization */
    InitInstance ( hInstance, nCmdShow, "title" ) ;

    /* message loop */
    while ( GetMessage ( &m, 0, 0, 0 ) )
        DispatchMessage ( &m ) ;

    return 0 ;
}

LRESULT CALLBACK WndProc ( HWND hWnd, UINT message,
                           WPARAM wParam, LPARAM lParam )
{
    switch ( message )
    {
        case WM_DESTROY :
            OnDestroy ( hWnd ) ;
            break ;
        default :
            return DefWindowProc ( hWnd, message, wParam, lParam ) ;
    }
    return 0 ;
}
```

```
void OnDestroy ( HWND hWnd )  
{  
    PostQuitMessage ( 0 );  
}
```

On execution of this program the window shown in Figure 17.4 appears on the screen. We can use minimize and the maximize button in its title bar to minimize and maximize the window. We can stretch its size by dragging its boundaries. Finally, we can close the window by clicking on the close window button in the title bar.



Figure 17.4

Let us now try to understand this program step by step.

Creation and Displaying of Window

Creating and displaying a window on the screen is a 4-step process. These steps are:

- (a) Creation of a window class.
- (b) Registering the window class with the OS.
- (c) Creation of a window based on the registered class.
- (d) Displaying the window on the screen.

Creation of a window class involves setting up of elements of a structure called **WNDCLASSEX**. This structure contains several

elements. They govern the properties of the window. Registration of a window class, creation of a window and displaying of a window involves calling of API functions **RegisterClassEx()**, **CreateWindow()** and **ShowWindow()** respectively. Since all the 4 steps mentioned above would be required in almost every program in this chapter I have written this code in a user-defined function called **InitInstance()** in the file ‘helper.h’.

Though writing code in a header file goes against the convention I have still done so to achieve simplicity. The complete listing of ‘helper.h’ file is available in Appendix F. Alternatively you can download it from the following link:

www.kicit.com/books/letusc/sourcecode/helper.h

As expected **WinMain()** starts off by calling the function **InitInstance()** present in ‘helper.h’ file. This file has been **#included** at the beginning of the program. Remember to copy this file to your project directory—the directory in which you are going to create this program.

Once the window has been created and displayed let us see how we can interact with it.

Interaction with Window

As and when the user interacts with the window—by stretching its boundaries or clicking the buttons in the title bar, etc. a suitable message is posted into the message queue of our application. Our application should now pick them up from the message queue and process them.

A message contains a message id and some other additional information about the message. For example, a mouse click message would contain additional information like handle to the window with which the user has interacted, the coordinates of

mouse cursor and the status of mouse buttons. Since it is difficult to memorize the message ids they have been suitably **#defined** in 'windows.h'. The message id and the additional information are stored in a structure called MSG.

In **WinMain()** this MSG structure is retrieved from the message queue by calling the API function **GetMessage()**. The first parameter passed to this function is the address of the **MSG** structure variable. **GetMessage()** would pick the message info from the message queue and place it in the structure variable passed to it. Don't bother about the other parameters right now.

After picking up the message from the message queue we need to process it. This is done by calling the **DispatchMessage()** API function. This function does several activities. These are as follows:

- (a) From the **MSG** structure that we pass to it, **DisplayMessage()** extracts the handle of the window for which this message is meant for.
- (b) From the handle it figures out the window class based on which the window has been created.
- (c) From the window class structure it obtains the address of a function called **WndProc()** (short for window procedure). Well I didn't tell you earlier that in **InitInstance()** while filling the **WNDCLASSEX** structure one of the elements has been set up with the address of a user-defined function called **WndProc()**.
- (d) Using this address it calls the function **WndProc()**.

Since several messages get posted into the message queue picking of the message and processing it should be done repeatedly. Hence calls to **GetMessage()** and **DispatchMessage()** have been made in a **while** loop in **WinMain()**. When **GetMessage()** encounters a message with id **WM_QUIT** it returns a **0**. Now the control comes out of the loop and **WinMain()** comes to an end.

Reacting to Messages

As we saw in the previous section, for every message picked up from the message queue the control is transferred to the **WndProc()** function. This function is shown below:

```
LRESULT CALLBACK WndProc ( HWND hWnd, UINT message,  
                           WPARAM wParam, LPARAM lParam )
```

This function always receives four parameters. The first parameter is the handle to the window for which the message has been received. The second parameter is the message id, whereas, the third and fourth parameters contain additional information about the message.

LRESULT is a **typedef** of a **long int** and represents the return value of this function. **CALLBACK** is a **typedef** of **__stdcall**. This **typedef** has been done in 'windows.h'. **CALLBACK** indicates that the **WndProc** function has been registered with Windows (through **WNDCLASSEX** structure in **InitInstance()**) with an intention that Windows would call this back (through **DispatchMessage()** function).

In the **WndProc()** function we have checked the message id using a **switch**. If the id is **WM_DESTROY** then we have called the function **OnDestroy()**. This message is posted to the message queue when the user clicks on the 'Close Window' button in the title bar. In **OnDestroy()** function we have called the API function **PostQuitMessage()**. This function posts a **WM_QUIT** message into the message queue. As we saw earlier, when this message is picked up the message loop and **WinMain()** is terminated.

For all messages other than **WM_DESTROY** the control lands in the **default** clause of **switch**. Here we have simply made a call to **DefWindowProc()** API function. This function does the default

processing of the message that we have decided not to tackle. The default processing for different message would be different. For example on double clicking the title bar **DefWindowProc()** maximizes the window.

Actually speaking when we close the window a **WM_CLOSE** message is posted into the message queue. Since we have not handled this message the **DefWindowProc()** function gets called to tackle this message. The **DefWindowProc()** function destroys the window and places a **WM_DESTROY** message in the message queue. As discussed earlier, in **WndProc()** we have made the provision to terminate the application on encountering **WM_DESTROY**.

That brings us to the end of a lonnnngggg explanation! You can now heave a sigh of relief. I would urge you to go through the above explanation till the time you are absolutely sure that you have understood every detail of it. A very clear understanding of it would help you make a good Windows programmer. For your convenience I have given a flowchart of the entire working in Figure 17.5.

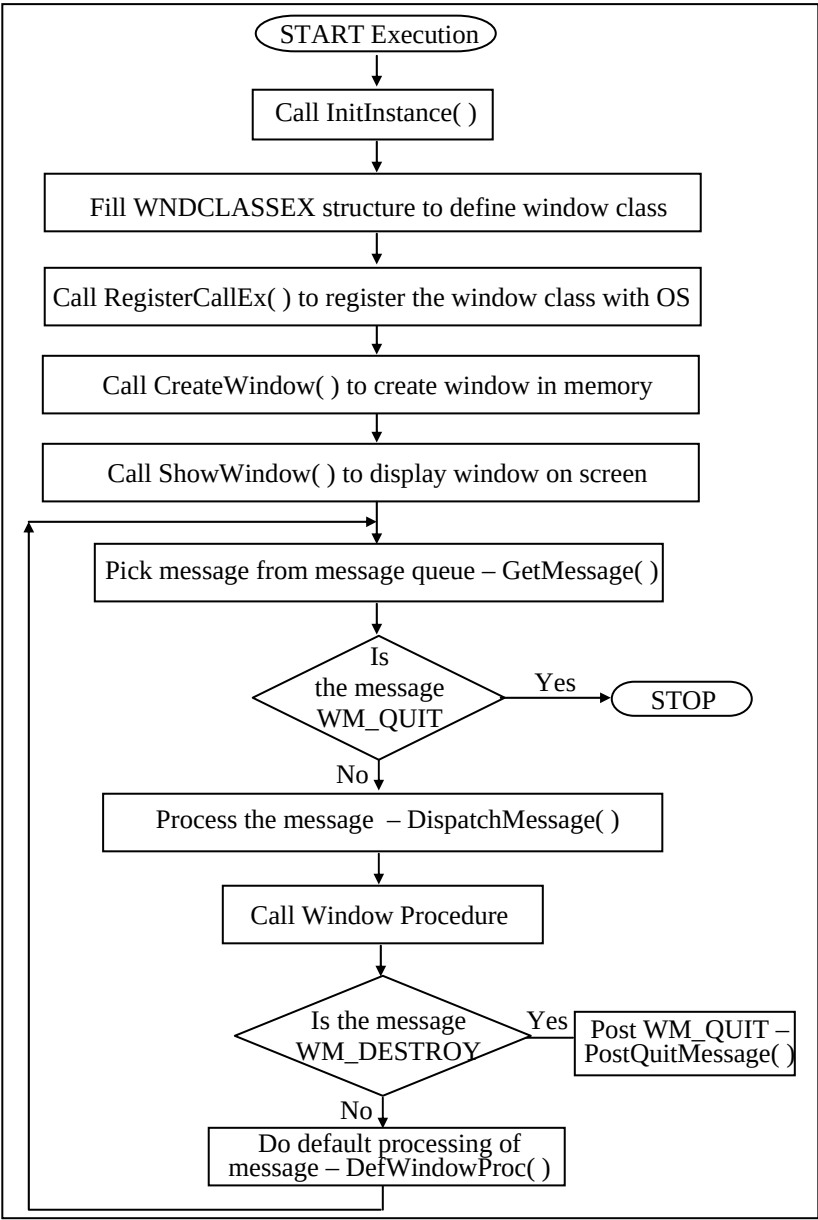


Figure 17.5

Program Instances

Windows allows you to run more than one copy of a program at a time. This is handy for cutting and pasting between two copies of Notepad or when running more than one terminal session with a terminal emulator program. Each running copy of a program is called a ‘program instance’.

Windows performs an interesting memory optimization trick. It shares a single copy of the program’s code between all running instances. For example, if you get three instances of Notepad running, there will only be one copy of Notepad’s code in memory. All three instances share the same code, but will have separate memory areas to hold the text data being edited. The difference between handling of the code and the data is logical, as each instance of Notepad might edit a different file, so the data must be unique to each instance. The program logic to edit the files is the same for every instance, so there is no reason why a single copy of Notepad’s code cannot be shared.

Summary

- (a) A message box can be displayed by calling the **MessageBox()** API function.
- (b) Message boxes are often used to ascertain the flow of a program.
- (c) Appearance of a message box can be customized.
- (d) The **CreateWindow()** API function creates the window in memory.
- (e) The window that is created in memory is displayed using the **ShowWindow()** API function.
- (f) A ‘window class’ specifies various properties of the window that we are creating.
- (g) The header file ‘Windows.h’ contains declaration of several macros used in Windows programming.

- (h) When the user clicks in a window, or moves mouse pointer on the window, messages are generated and posted in the application message queue.
- (i) A message contains the message id and additional information about the message.
- (j) The **GetMessage()-DispatchMessage()** loop breaks when **GetMessage()** encounters the **WM_QUIT** message.
- (k) If we don't handle a message received by our application then the **DefWindowProc()** function is called to do the default processing.

Exercise

[A] State True or False:

- (a) **MessageBox()** is an API function.
- (b) Calling the **MessageBox()** function displays the specified string in console window.
- (c) The **CreateWindow()** function creates and displays the window on the screen.
- (d) The **ShowWindow()** function can display only the maximized window.
- (e) Every window has to be created using pre-registered window class.
- (f) Window classes are similar to classes in C++.
- (g) We can use the pre-defined window classes but cannot create our own.
- (h) The style **WS_OVERLAPPED | WS_CAPTION | WS_MINIMIZEBOX** will create a window with caption bar and minimize box only.
- (i) To be able to interact with a window it is necessary to implement the message loop.

[B] Answer the following:

- (a) Outline the steps that a typical Windows program follows during execution.

- (b) Run any Windows based program and see whether you can identify all the elements of the application window.
- (c) How would you minimize a window programmatically?
- (d) What would happen if we do not place WM_QUIT message in the message queue when the user tries to close the window.
- (e) Explain the need of **RegisterClassEx()** function.
- (f) What is the difference between **GetMessage()** and **DispatchMessage()** function?
- (g) Write a program, which receives an integer as a command line argument, creates a button window, and based on the value of the integer displays button window as maximized / minimized / normal.
- (h) Try to display a window with different combinations of window styles and observe the results.

18 Graphics Under Windows

- Graphics as of Now
- Device Independent Drawing
- Hello Windows
- Drawing Shapes
- Types of Pens
- Types of Brushes
 - Code and Resources
- Freehand Drawing, the Paintbrush Style
- Capturing the mouse
- Device Context, A Closer Look
- Displaying a Bitmap
- Animation at Work
 - WM_CREATE and *OnCreate()*
 - WM_TIMER and *OnTimer()*
 - A Few More Points...
- Windows, the Endless World...
- Summary
- Exercise

Since times immemorial colors and shapes have fascinated mankind like nothing else. Otherwise people would have still been using the character oriented interfaces of MS-DOS or Unix. In fact the graphical ability of Windows has played a very important role in its success story. Once you get a hang of how to draw inside a window it would open up immense possibilities that you never thought were possible.

Graphics as of Now

World has progressed much beyond 16 colors and 640 x 480 resolution graphics that Turbo C/C++ compilers offered under MS-DOS environment. Today we are living in a world of 1024 x 768 resolution offering 16.7 million colors. Graphical menus, icons, colored cursors, bitmaps, wave files and animations are the order of the day. So much so that a 16-color graphics program built using Turbo C working on a poor resolution almost hurts the eye. Moreover, with the whole lot of Windows API functions to support graphics activity there is so much that can be achieved in a graphics program under Windows. I am sure that this chapter will help you understand and appreciate these new capabilities.

Device Independent Drawing

Windows allow programmers to write programs to display text or graphics on the screen without concern over the specifics of the video hardware. A Windows program that works on a VGA display will work without modification on an SVGA or on a XGA display that Windows supports.

The key to this 'device independence' is Windows' use of a 'device context'. We will explore how the device context can be used for both text and graphics output, and how using the device context keeps our programs from interfering with each other on the screen.

During the original design of Windows, one of the goals was to provide ‘device independence’. Device independence means that the same program should be able to work using different screens, keyboards and printers without modification to the program. Windows takes care of the hardware, allowing the programmer to concentrate on the program itself. If you have ever had to update the code of an MS-DOS program for the latest printer, plotter, video display, or keyboard, you will recognize device independence as a huge advantage for the developer.

Windows programs do not send data directly to the screen or printer. A Windows program knows where (screen/printer) its output is being sent. However, it does not know how it would be sent there, neither does it need to bother to know this. This is because Windows uses a standard and consistent way to send the output to screen/printer. This standard way uses an entity called Device Context, or simply a DC. Different DC’s are associated with different devices. For example, a screen DC is associated with a screen, a printer DC is associated with a printer, etc. Any drawing that we do using the screen DC is directed to the screen. Similarly, any drawing done using the printer DC is directed to the printer. Thus, the only thing that changes from drawing to screen and drawing to printer is the DC that is used.

A windows program obtains a handle (ID value) for the screen or printer’s DC. The output data is sent to the screen/printer using its DC, and then Windows and the Device Driver for the device takes care of sending it to the real hardware. The advantage of using the DC is that the graphics and text commands that we send using the DC are always the same, regardless of where the physical output is showing up.

The part of Windows that converts the Windows graphics function calls to the actual commands sent to the hardware is the GDI, or Graphics Device Interface. The GDI is a program file called GDI32.DLL and is stored in the Windows System directory. The

Windows environment loads GDI32.DLL into memory when it is needed for graphical output. Windows also loads a ‘device driver’ program if the hardware conversions are not part of GDI32.DLL. Common examples are VGA.SYS for VGA video screen and HPPLC.SYS for the HP LaserJet printer. Drivers are just programs that assist the GDI in converting Windows graphics commands to hardware commands.

Thus GDI provides all the basic drawing functionality for Windows; the device context represents the device providing a layer of abstraction that insulates your applications from the trouble of drawing directly to the hardware. The GDI provides this insulation by calling the appropriate device driver in response to windows graphics function calls.

Hello Windows

We would begin our tryst with graphics programming under windows by displaying a message “Hello Windows” in different fonts. Note that though we are displaying text under Windows even text gets drawn graphically in the window. First take a look at the program given below before we set out to understand it.

```
# include <windows.h>
# include "helper.h"

void OnPaint ( HWND ) ;
void OnDestroy ( HWND ) ;

int __stdcall WinMain ( HINSTANCE hInstance, HINSTANCE hPrevInstance,
LPSTR lpszCmdline, int nCmdShow )
{
    MSG m ;

    /* Perform application initialization */
    InitInstance ( hInstance, nCmdShow, "Text" ) ;
```



```
/* Main message loop */
while ( GetMessage ( &m, NULL, 0, 0 ) )
    DispatchMessage(&m);

return 0 ;
}

LRESULT CALLBACK WndProc ( HWND hWnd, UINT message,
                          WPARAM wParam, LPARAM lParam )
{
    switch ( message )
    {
        case WM_DESTROY :
            OnDestroy ( hWnd ) ;
            break ;
        case WM_PAINT :
            OnPaint ( hWnd ) ;
            break ;
        default :
            return DefWindowProc ( hWnd, message, wParam, lParam ) ;
    }
    return 0 ;
}

void OnDestroy ( HWND hWnd )
{
    PostQuitMessage ( 0 ) ;
}

void OnPaint ( HWND hWnd )
{
    HDC hdc ;
    PAINTSTRUCT ps ;
    HFONT hfont ;
    LOGFONT f = { 0 } ;
    HGDIOBJ holdfont ;
    char *fonts[ ] = { "Arial", "Times New Roman", "Comic Sans MS" } ;
    int i ;
```

```
hdc = BeginPaint ( hWnd, &ps ) ;

for ( i = 0 ; i < 3 ; i++ )
{
    strcpy ( f.lfFaceName, fonts[ i ] ) ; /* copy font name */
    f.lfHeight = 40 * ( i + 1 ) ; /* font height */
    f.lfItalic = 1 ; /* italic */

    hfont = CreateFontIndirect ( &f ) ;
    holdfont = SelectObject ( hdc, hfont ) ;

    SetTextColor ( hdc, RGB ( 0, 0, 255 ) ) ;

    TextOut ( hdc, 10, 70 * i, "Hello Windows", 13 ) ;

    SelectObject ( hdc, holdfont ) ;
    DeleteObject ( hfont ) ;
}

EndPaint ( hWnd, &ps ) ;
}
```

On execution of this program the window shown in Figure 18.1 appears.



Figure 18.1

Drawing to a window involves handling the **WM_PAINT** message. This message is generated whenever the client area of the window needs to be redrawn. This redrawing would be required in the following situations:

- (a) When the Window is displayed for the first time.
- (b) When the window is minimized and then maximized.
- (c) When some portion of the window is overlapped by another window and the overlapped window is dismissed.
- (d) When the size of the window changes on stretching its boundaries.
- (e) When the window is dragged out of the screen and then brought back into the screen.

Would a **WM_PAINT** message be generated when the cursor is dragged in the window? No. In this case the window saves the area overlapped by the cursor and restores it when the cursor moves to another position.

When the **switch-case** structure inside **WndProc()** finds that the message ID passed to **WndProc()** is **WM_PAINT**, it calls the function **OnPaint()**. Within **OnPaint()** we have called the API function **BeginPaint()**. This function obtains a handle to the device context. Additionally it also fills the **PAINTSTRUCT** structure with information about the area of the window which needs to be repainted. Lastly it removes **WM_PAINT** from the message queue. After obtaining the device context handle, the control enters a loop.

Inside the loop we have displayed “Hello Windows” in three different fonts. Each time through the loop we have setup a **LOGFONT** structure **f**. This structure is used to indicate the font properties like font name, font height, italic or normal, etc. Note that in addition to these there are other font properties that may be setup. The properties that we have not setup in the loop are all initialized to **0**. Once the font properties have been setup we have called the **CreateFontIndirect()** API function to create the font.

This function loads the relevant font file. Then using the information in the font file and the font properties setup in the **LOGFONT** structure it creates a font in memory. **CreateFontIndirect()** returns the handle to the font created in memory. This handle is then passed to the **SelectObject()** API function to get the font into the DC. This function returns the handle to the existing font in the DC, which is preserved in **hfont** variable. Next we have used the **SetTextColor()** API function to set the color of the text to be displayed through **TextOut()**. The **RGB()** macro uses the red, green and blue component values to generate a 32-bit color value. Note that each color component can take a value from **0** to **255**. To **TextOut()** we have to pass the handle to the DC, position where the text is to be displayed, the text to be displayed and its length.

With **hfont** only one font can be associated at a time. Hence before associating another font with it we have deleted the existing font using the **DeleteObject()** API function. Once outside the loop we have called the **EndPaint()** API function to release the DC handle. If not released we would be wasting precious memory, because the device context structure would remain in memory but we would not be able access it.

In place of **TextOut()** we can also use the **DrawText()** API function. This function permits finer control over the way the text is displayed. You can explore this function on your own.

Drawing Shapes

If text is so near can graphics be far behind? Now that we know how to draw text in a window let us now create a simple program that displays different shapes in a window. Instead of showing the entire program given below is the listing of **OnPaint()**. The rest of the program is same as in the previous section. Here onwards I would be showing only the **OnPaint()** handler unless otherwise required.

```
void OnPaint ( HWND hWnd )
{
    HDC hdc ;
    PAINTSTRUCT ps ;
    HBRUSH hbr ;
    HGDIOBJ holdbr ;
    POINT pt[5] = { 250, 150, 250, 300, 300, 350, 400, 300, 320, 190 } ;

    hdc = BeginPaint ( hWnd, &ps ) ;

    hbr = CreateSolidBrush ( RGB ( 255, 0, 0 ) ) ;
    holdbr = SelectObject ( hdc, hbr ) ;

    MoveToEx ( hdc, 10, 10, NULL ) ;
    LineTo ( hdc, 200, 10 ) ;

    Rectangle ( hdc, 10, 20, 200, 100 ) ;

    RoundRect ( hdc, 10, 120, 200, 220, 20, 20 ) ;

    Ellipse ( hdc, 10, 240, 200, 340 ) ;

    Pie ( hdc, 250, 10, 350, 110, 350, 110, 350, 10 ) ;

    Polygon ( hdc, pt, 5 ) ;

    SelectObject ( hdc, holdbr ) ;
    DeleteObject ( hbr ) ;

    EndPaint ( hWnd, &ps ) ;
}
```

On execution of this program the window shown in Figure 18.2 appears.

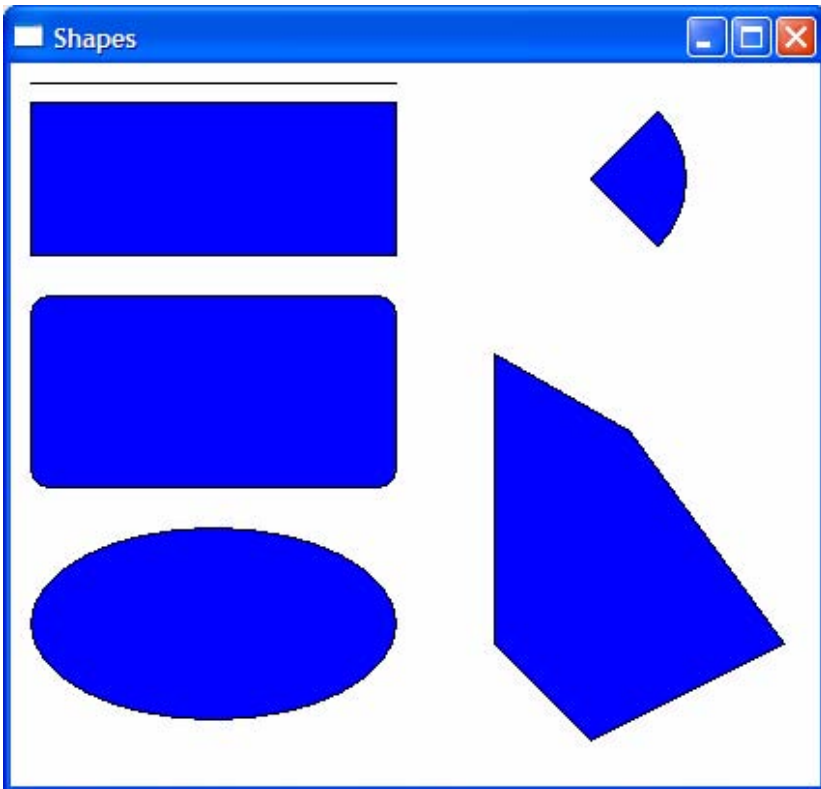


Figure 18.2

For drawing any shape we need a pen to draw its boundary and a brush to paint the area enclosed by it. The DC contains a default pen and brush. The default pen is a solid pen of black color and the default brush is white in color. In this program we have used the default pen and a blue colored solid brush for drawing the shapes.

As before, we begin by obtaining a handle to the DC using **BeginPaint()** function. For creating a solid colored brush we need to call the **CreateSolidBrush()** API function. The second parameter of this function specifies the color of the brush. The function returns the handle of the brush which we have preserved

in the **hbr** variable. Next we have selected this brush in the DC. The handle of the default brush in DC is collected in the **holdbr** variable.

Once we have selected the brush into the DC we are ready to draw the shapes. For drawing the line we have used **MoveToEx()** and **LineTo()** API functions. Similarly for drawing a rectangle we have used the **Rectangle()** function.

The **RoundRect()** function draws a rectangle with rounded corners. In **RoundRect (x1, y1, x2, y2, x3, y3)**, **x1, y1** represents the *x* and *y*-coordinates of the upper-left corner of the rectangle. Likewise, **x2, y2** represent coordinates of the bottom right corner of the rectangle. **x3, y3** specify the width and height of the ellipse used to draw the rounded corners.

Note that rectangle and the rounded rectangle are drawn from **x1, y1** up to **x2-1, y2-1**.

Parameters of **Ellipse()** specify coordinates of bounding rectangle of the ellipse.

The **Pie()** function draws a pie-shaped wedge by drawing an elliptical arc whose center and two endpoints are joined by lines. The center of the arc is the center of the bounding rectangle specified by **x1, y1** and **x2, y2**. In **Pie(x1, y1, x2, y2, x3, y3, x4, y4)**, **x1, y1** and **x2, y2** specify the *x* and *y*-coordinates of the upper left corner and bottom right corner respectively, of the bounding rectangle. **x3, y3** and **x4, y4** specify the *x* and *y*-coordinates of the arc's starting point and ending point respectively.

In **Polygon (lpPoints, nCount)**, **lpPoints** points to an array of points that specifies the vertices of the polygon. Each point in the array is a **POINT** structure. **nCount** specifies the number of vertices stored in the array. The system closes the polygon automatically, if necessary, by drawing a line from the last vertex to the first.

Once we are through with drawing the shapes the old brush is selected back in the DC and then the brush created by us is deleted using **DeleteObject()** function.

Types of Pens

In the previous program we have used the default solid black pen of thickness 1 pixel. We can create pens of different style, color and thickness to do our drawing. The following **OnPaint()** handler shows how this can be achieved.

```
void OnPaint ( HWND hWnd )
{
    HDC hdc ;
    PAINTSTRUCT ps ;
    HPEN hpen ;
    HGDIOBJ holdpen ;

    hdc = BeginPaint ( hWnd, &ps ) ;

    hpen = CreatePen ( PS_DASH, 1, RGB ( 255, 0, 0 ) ) ;
    holdpen = SelectObject ( hdc, hpen ) ;

    MoveToEx ( hdc, 10, 10, NULL ) ;
    LineTo ( hdc, 500, 10 ) ;

    SelectObject ( hdc, holdpen ) ;
    DeleteObject ( hpen ) ;

    hpen = CreatePen ( PS_DOT, 1, RGB ( 255, 0, 0 ) ) ;
    holdpen = SelectObject ( hdc, hpen ) ;

    MoveToEx ( hdc, 10, 60, NULL ) ;
    LineTo ( hdc, 500, 60 ) ;

    SelectObject ( hdc, holdpen ) ;
    DeleteObject ( hpen ) ;
}
```



```
hpen = CreatePen ( PS_DASHDOT, 1, RGB ( 255, 0, 0 ) );
holdpen = SelectObject ( hdc, hpen );

MoveToEx ( hdc, 10, 110, NULL );
LineTo ( hdc, 500, 110 );

SelectObject ( hdc, holdpen );
DeleteObject ( hpen );

hpen = CreatePen ( PS_DASHDOTDOT, 1, RGB ( 255, 0, 0 ) );
holdpen = SelectObject ( hdc, hpen );

MoveToEx ( hdc, 10, 160, NULL );
LineTo ( hdc, 500, 160 );

SelectObject ( hdc, holdpen );
DeleteObject ( hpen );

hpen = CreatePen ( PS_SOLID, 10, RGB ( 255, 0, 0 ) );
holdpen = SelectObject ( hdc, hpen );

MoveToEx ( hdc, 10, 210, NULL );
LineTo ( hdc, 500, 210 );

SelectObject ( hdc, holdpen );
DeleteObject ( hpen );

EndPoint ( hWnd, &ps );
}
```

On execution of this program the window shown in Figure 18.3 appears.



Figure 18.3

A new pen can be created using the **CreatePen()** API function. This function needs three parameters—pen style, pen thickness and pen color. Different macros like PS_SOLID, PS_DOT, etc. have been defined in ‘windows.h’ to represent different pen styles. Note that for pen styles other than PS_SOLID the pen thickness has to be 1 pixel.

Types of Brushes

The way we can create different types of pens, we can also create three different types of brushes. These are—solid brush, hatch brush and pattern brush. Let us now write a program that shows how to build these brushes and then use them to fill rectangles. Here is the **OnPaint()** handler which achieves this.

```
void OnPaint ( HWND hWnd )  
{  
    HDC hdc ;  
    PAINTSTRUCT ps ;  
    HBRUSH hbr ;
```

```
HGDIOBJ holdbr ;
HBITMAP hbmp ;

hdc = BeginPaint ( hWnd, &ps ) ;

hbr = CreateSolidBrush ( RGB (255, 0, 0) ) ;
holdbr = SelectObject ( hdc, hbr ) ;

Rectangle ( hdc, 5, 5, 105, 100 ) ;

SelectObject ( hdc, holdbr ) ;
DeleteObject ( hbr ) ;

hbr = CreateHatchBrush ( HS_CROSS, RGB ( 255, 0, 0 ) ) ;
holdbr = SelectObject ( hdc, hbr ) ;

Rectangle ( hdc, 125, 5, 225, 100 ) ;

SelectObject ( hdc, holdbr ) ;
DeleteObject ( hbr ) ;

hbmp = LoadBitmap ( hInst, MAKEINTRESOURCE ( IDB_BITMAP1 ) ) ;

hbr = CreatePatternBrush ( hbmp ) ;
holdbr = SelectObject ( hdc, hbr ) ;

Rectangle ( hdc, 245, 5, 345, 100 ) ;

SelectObject ( hdc, holdbr ) ;
DeleteObject ( hbr ) ;
DeleteObject ( hbmp ) ;

EndPaint ( hWnd, &ps ) ;

DeleteObject ( hbr ) ;
}
```

On execution of this program the window shown in Figure 18.4 appears.

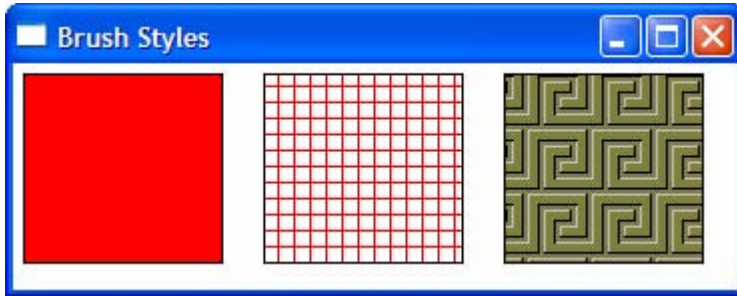


Figure 18.4

In the **OnPaint()** handler we have drawn three rectangles—first using a solid brush, second using a hatched brush and third using a pattern brush. Creating and using a solid brush and hatched brush is simple. We simply have to make calls to **CreateSolidBrush()** and **CreateHatchBrush()** respectively. For the hatch brush we have used the style `HS_CROSS`. There are several other styles defined in ‘windows.h’ that you can experiment with.

For creating a pattern brush we need to first create a bitmap (pattern). Instead of creating this pattern, we have used a readymade bitmap file. You can use any other bitmap file present on your hard disk.

Bitmaps, menus, icons, cursors that a Windows program may use are its resources. When the compile such a program we usually want these resources to become a part of our EXE file. If so done we do not have to ship these resources separately. To be able to use a resource (bitmap file in our case) it is not enough to just copy it in the project directory. Instead we need to carry out the steps mentioned below to add a bitmap file to the project.

- (a) From the ‘Insert’ menu option of VC++ 6.0 select the ‘Resource’ option.

- (b) From the dialog that pops up select 'bitmap' followed by the import button.
- (c) Select the suitable .bmp file.
- (d) From the 'File' menu select the save option to save the generated resource script file (Script1.rc). When we select 'Save' one more file called 'resource.h' also gets created.
- (e) Add the 'Script1.rc' file to the project using the Project | Add to Project | Files option.

While using the bitmap in the program it is always referred using an id. The id is **#defined** in the file 'resource.h'. Somewhere information has to be stored linking the id with the actual .bmp file on the disk. This is done in the 'Script1.rc' file. We need to include the 'resource.h' file in the program.

To create the pattern brush we first need to load the bitmap in memory. We have done this using the **LoadBitmap()** API function. The first parameter passed to this function is the handle to the instance of the program. When **InitInstance()** function is called from **WinMain()** it stores the instance handle in a global variable **hInst**. We have passed this **hInst** to **LoadBitmap()**. The second parameter passed to it is a string representing the bitmap. This string is created from the resource id using the **MAKEINTRESOURCE** macro. The **LoadBitmap()** function returns the handle to the bitmap. This handle is then passed to the **CreatePatternBrush()** function. This brush is then selected into the DC and then a rectangle is drawn using it.

Note that if the size of the bitmap is bigger than the rectangle being drawn then the bitmap is suitably clipped. On the other hand if the bitmap is smaller than the rectangle it is suitably replicated.

While doing the clean up firstly the brush is deleted followed by the bitmap.

Code and Resources

A program consists of both instructions and static data. Static data is that portion of the program which is not executed as machine instructions and which does not change as the program executes. Static data are character strings, data to create fonts, bitmaps, etc. The designers of Windows wisely decided that static data should be handled separately from the program code. The Windows term for static data is 'Resource data', or simply 'Resources'. By separating static data from the program code the creators of Windows were able to use a standard C/C++ compiler to create the code portion of the finished Windows program, and they only had to write a 'Resource compiler' to create the resources that Windows programs use. Separating the code from the resource data has other advantages like reducing memory demands and making programs more portable. It also means that a programmer can work on a program's logic, while a designer works on how the program looks.

Freehand Drawing, the Paintbrush Style

Even if you are knee high in computers I am sure you must have used PaintBrush. It provides a facility to draw a freehand drawing using mouse. Let us see if we too can achieve this. We can indicate where the freehand drawing begins by clicking the left mouse button. Then as we move the mouse on the table with the left mouse button depressed the freehand drawing should get drawn in the window. This drawing should continue till we do not release the left mouse button.

The mouse input comes in the form of messages. For free hand drawing we need to tackle three mouse messages—**WM_LBUTTONDOWN** for left button click, **WM_MOUSEMOVE** for mouse movement and **WM_LBUTTONUP** for releasing the left mouse button. Let us now see how these messages are tackled for drawing freehand. The

WndProc() function and the message handlers that perform this task are given below

```
int x1, y1, x2, y2 ;
```

```
LRESULT CALLBACK WndProc ( HWND hWnd, UINT message,
                           WPARAM wParam, LPARAM lParam )
{
    switch ( message )
    {
        case WM_DESTROY :
            OnDestroy ( hWnd ) ;
            break ;

        case WM_LBUTTONDOWN :
            OnLButtonDown ( hWnd, LOWORD ( lParam ),
                           HIWORD ( lParam ) ) ;

            break ;

        case WM_LBUTTONUP :
            OnLButtonUp ( ) ;
            break ;

        case WM_MOUSEMOVE :
            OnMouseMove ( hWnd, wParam, LOWORD ( lParam ),
                           HIWORD ( lParam ) ) ;

            break ;

        default:
            return DefWindowProc ( hWnd, message, wParam, lParam ) ;
    }
    return 0 ;
}
```

```
void OnLButtonDown ( HWND hWnd, int x, int y )
{
    SetCapture ( hWnd ) ;
    x1 = x ;
```

```
    y1 = y ;
}

void OnMouseMove ( HWND hWnd, int flags, int x, int y )
{
    HDC hdc ;
    if ( flags == MK_LBUTTON ) /* is left mouse button depressed */
    {
        hdc = GetDC ( hWnd ) ;
        x2 = x ;
        y2 = y ;
        MoveToEx ( hdc, x1, y1, NULL ) ;
        LineTo ( hdc, x2, y2 ) ;

        ReleaseDC ( hWnd, hdc ) ;

        x1 = x2 ;
        y1 = y2 ;
    }
}

void OnLButtonUp( )
{
    ReleaseCapture( ) ;
}
```

On execution of this program the window shown in Figure 18.5 appears. We can now click the left mouse button with mouse pointer placed anywhere in the window. We can then drag the mouse on the table to draw the freehand. The freehand drawing would continue till we do not release the left mouse button.

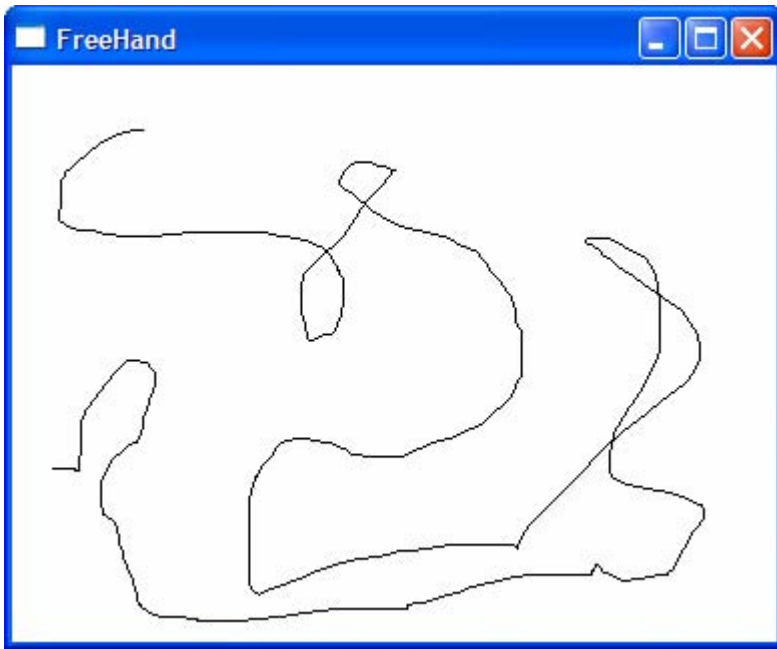


Figure 18.5

It appears that for drawing the freehand we should simply receive the mouse coordinates as it is moved and then highlight the pixels at these coordinates using the **SetPixel()** API function. However, if we do so the freehand would be broken at several places. This is because usually the mouse is dragged pretty fast whereas the mouse move messages won't arrive so fast. A solution to this problem is to construct the freehand using small little line segments. This is what has been done in our program. These lines are so small in size that you would not even recognize that the freehand has been drawn by connecting these small lines.

Let us now discuss each mouse handler. When the **WM_LBUTTONDOWN** message arrives the **WndProc()** function calls the handler **OnLButtonDown()**. While doing so, we have passed the mouse coordinates where the click occurred. These coordinates are obtained in **lParam** in **WndProc()**. In **lParam** the low order 16 bits contain the current x - coordinate of the mouse whereas the high order 16 bits contain the y - coordinate. The **LOWORD** and **HIWORD** macros have been used to separate out these x and y - coordinates from **lParam**.

In **OnLButtonDown()** we have preserved the starting point of freehand in global variables **x1** and **y1**.

When **OnMouseMove()** gets called it checks whether the left mouse button stands depressed. If it stands depressed then the **flags** variable contains **MK_LBUTTON**. If it does, then the current mouse coordinates are set up in the global variables **x2, y2**. A line is then drawn between **x1, y1** and **x2, y2** using the functions **MoveToEx()** and **LineTo()**. Next time around **x2, y2** should become the starting of the next line. Hence the current values of **x2, y2** are stored in **x1, y1**.

Note that here we have obtained the DC handle using the API function **GetDC()**. This is because we are carrying out the drawing activity in reaction to a message other than **WM_PAINT**. Also, the handle obtained using **GetDC()** should be released using a call to **ReleaseDC()** function.

You can try using **BeginPaint() / EndPaint()** in mouse handlers and **GetDC() / ReleaseDC()** in **OnPaint()**. Can you draw any conclusions?

Capturing the Mouse

If in the process of drawing the freehand the mouse cursor goes outside the client area then the window below our window would

start getting mouse messages. So our window would not receive any messages. If this has to be avoided then we should ensure that our window continues to receive mouse messages even when the cursor goes out of the client area of our window. The process of doing this is known as mouse capturing.

We have captured the mouse in **OnLButtonDown()** handler by calling the API function **SetCapture()**. As a result, the program continues to respond to mouse events during freehand drawing even if the mouse is moved outside the client area. In the **OnLButtonUp()** handler we have released the captured mouse by calling the **ReleaseCapture()** API function.

Device Context, a Closer Look

Now that we have written a few programs and are comfortable with idea of selecting objects like font, pen and brush into the DC, it is time for us to understand how Windows achieves the device independent drawing using the concept of DC. In fact a DC is nothing but a structure that holds handles of various drawing objects like font, pen, brush, etc. A screen DC and its working is shown in Figure 18.6.

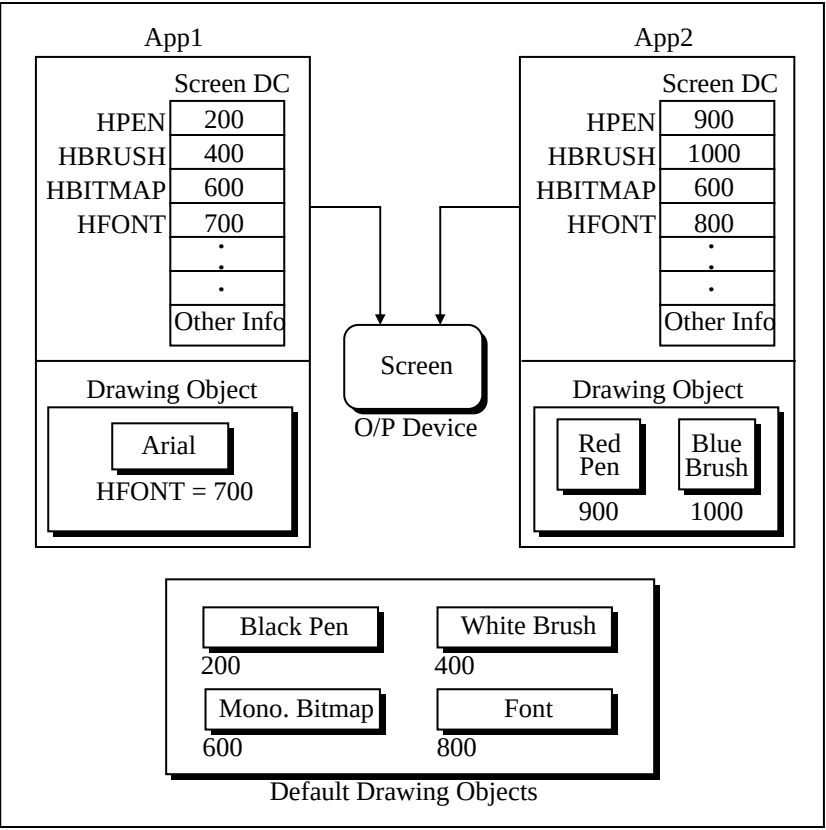


Figure 18.6

You can make following observations from Figure 18.6:

- (a) The DC doesn't hold the drawing objects like pen, brush, etc. It merely holds their handles.
- (b) With each DC a default monochrome bitmap of size 1 pixel x 1 pixel is associated.
- (c) Default objects like black pen, white brush, etc. are shared by different DCs in same or different applications.

- (d) The drawing objects that an application explicitly creates can be shared within DCs of the same application, but is never shared between different applications.
- (e) Two different applications would need two different DCs even though both would be used to draw to the same screen. In other words with one screen multiple DCs can exist.
- (f) A common Device Driver would serve the drawing requests coming from different applications. (Truly speaking the request comes from GDI functions that our application calls).

Screen and printer DC is OK, but what purpose would a memory DC serve? Well, that is what the next program would explain.

Displaying a Bitmap

We are familiar with drawing normal shapes on screen using a device context. How about drawing images on the screen? Windows does not permit displaying a bitmap image directly using a screen DC. This is because there might be color variations in the screen on which the bitmap was created and the screen on which it is being displayed. To account for such possibilities while displaying a bitmap Windows uses a different mechanism—a ‘Memory DC’

The way anything drawn using a screen DC goes to screen, anything drawn using a printer DC goes to a printer, similarly anything drawn using a memory DC goes to memory (RAM). But where in RAM—in the 1 x 1 pixel bitmap whose handle is present in memory DC. (Note that this handle was of little use in case of screen/printer DC). Thus if we attempt to draw a line using a memory DC it would end up on the 1 x 1 pixel bitmap. You would agree 1 x 1 is too small a place to draw even a small line. Hence we need to expand the size and color capability of this bitmap. How can this be done? Simple, just replace the handle of the 1 x 1 bitmap with the handle of a bigger and colored bitmap object. This is shown in Figure 18.7.

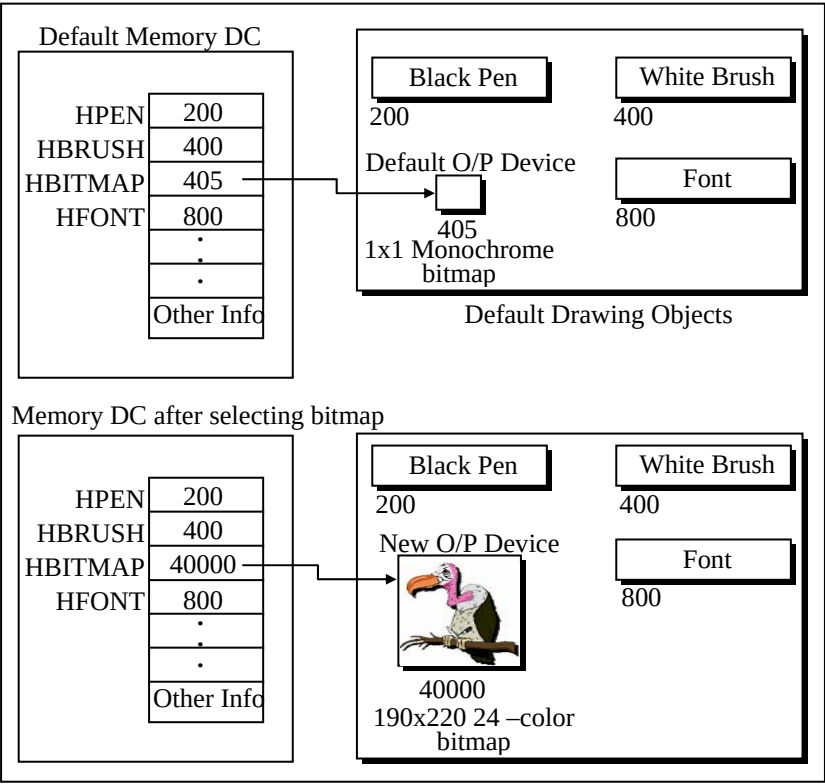


Figure 18.7

What purpose would just increasing the bitmap size/color would serve? Whatever we draw here would get drawn on the bitmap but would still not be visible. We can make it visible by simply copying the bitmap image (including what has been drawn on it) to the screen DC by using the API function **BitBlt()**.

Before transferring the image to the screen DC we need to make the memory DC compatible with the screen DC. Here making compatible means making certain adjustments in the contents of the memory DC structure. Looking at these values the screen device driver would suitably adjust the colors when the pixels in

the bitmap of memory DC is transferred to screen DC using **BitBlt()** function.

Let us now take a look at the program that puts all these concepts in action. The program merely displays the image of a vulture in a window. Here is the code...

```
void OnPaint ( HWND hWnd )
{
    HDC hdc ;
    HBITMAP hbmp ;
    HDC hmemdc ;
    HGDIOBJ holdbmp ;
    PAINTSTRUCT ps ;

    hdc = BeginPaint ( hWnd, &ps ) ;

    hbmp = LoadBitmap ( hInst, MAKEINTRESOURCE ( IDB_BITMAP1 ) ) ;

    hmemdc = CreateCompatibleDC ( hdc ) ;
    holdbmp = SelectObject ( hmemdc, hbmp ) ;

    BitBlt ( hdc, 10, 20, 190, 220, hmemdc, 0, 0, SRCCOPY ) ;

    EndPaint ( hWnd, &ps ) ;

    SelectObject ( hmemdc, holdbmp ) ;
    DeleteObject ( hbmp ) ;
    DeleteDC ( hmemdc ) ;
}
```

On executing the program we get the window shown in Figure 18.7.



Figure 18.7

As usual we begin our drawing activity in **OnPaint()** by first getting the screen DC using the **BeginPaint()** function. Next we have loaded the vulture bitmap image in memory by calling the **LoadBitmap()** function. Its usage is similar to what we saw while creating a pattern brush in an earlier section of this chapter. Then we have created a memory device context and made its properties compatible with that of the screen DC. To do this we have called the API function **CreateCompatibleDC()**. Note that we have passed the handle to the screen DC to this function. The function in turn returns the handle to the memory DC. After this we have selected the loaded bitmap into the memory DC. Lastly, we have performed a bit block transfer (a bit by bit copy) from memory DC to screen DC using the function **BitBlt()**. As a result of this the vulture now appears in the window.

We have made the call to **BitBlt()** as shown below:

```
BitBlt ( hdc, 10, 20, 190, 220, hmemdc, 0, 0, SRCCOPY );
```


Let us now understand its parameters. These are as under:

hdc – Handle to target DC where the bitmap is to be blitted

10, 20 – Position where the bitmap is to be blitted

190, 220 – Width and height of bitmap being blitted

0, 0 – Top left corner of the source image. If we give 10, 20 then the image from 10, 20 to bottom right corner of the bitmap would get blitted.

SRCCOPY – Specifies one of the raster-operation codes. These codes define how the color data for the source rectangle is to be combined with the color data for the destination rectangle to achieve the final color. SRCCOPY means that the pixel color of source should be copied onto the destination pixel of the target.

Animation at Work

Speed is the essence of life. So having the ability to display a bitmap in a window is fine, but if we can add movement and sound to it then nothing like it. So let us now see how to achieve this animation and sound effect.

If we are to animate an object in the window we need to carry out the following steps:

- (a) Create an image that is to be animated as a resource.
- (b) Prepare the image for later display.
- (c) Repeatedly display this prepared image at suitable places in the window taking care that when the next image is displayed the previous image is erased.
- (d) Check for collisions while displaying the prepared image.

Let us now write a program that on execution makes a red colored ball move in the window. As the ball strikes the walls of the

window a noise occurs. Note that the width and height of the red-colored ball is 22 pixels. Given below is the **WndProc()** function and the various message handlers that help achieve animation and sound effect.

```

HBITMAP hbmp ;
int x, y ;
HDC hmemdc ;
HGDIOBJ holdbmp ;

LRESULT CALLBACK WndProc ( HWND hWnd, UINT message,
                           WPARAM wParam, LPARAM lParam )
{
    switch ( message )
    {
        case WM_DESTROY :
            OnDestroy ( hWnd ) ;
            break ;
        case WM_CREATE :
            OnCreate ( hWnd ) ;
            break ;
        case WM_TIMER :
            OnTimer ( hWnd ) ;
            break ;
        default :
            return DefWindowProc ( hWnd, message, wParam, lParam ) ;
    }
    return 0 ;
}

void OnCreate ( HWND hWnd )
{
    RECT r ;
    HDC hdc ;

    hbmp = LoadBitmap ( hInst, MAKEINTRESOURCE ( IDB_BITMAP1 ) ) ;

    hdc = GetDC ( hWnd ) ;

```

```
hmemdc = CreateCompatibleDC ( hdc ) ;
holdbmp = SelectObject ( hmemdc, hbmp ) ;

ReleaseDC ( hWnd, hdc ) ;

srand ( time ( NULL ) ) ;

GetClientRect ( hWnd, &r ) ;

x = rand() % r.right - 22 ;
y = rand() % r.bottom - 22 ;

SetTimer ( hWnd, 1, 50, NULL ) ;
}

void OnDestroy ( HWND hWnd )
{
    KillTimer ( hWnd, 1 ) ;
    SelectObject ( hmemdc, holdbmp ) ;
    DeleteDC ( hmemdc ) ;
    DeleteObject ( hbmp ) ;
    PostQuitMessage ( 0 ) ;
}

void OnTimer ( HWND hWnd )
{
    HDC hdc ;
    RECT r ;
    const int wd = 22, ht = 22 ;
    static int dx = 10, dy = 10 ;

    hdc = GetDC ( hWnd ) ;
    BitBlt ( hdc, x, y, wd, ht, hmemdc, 0, 0, WHITENESS ) ;
    GetClientRect ( hWnd, &r ) ;

    x += dx ;
    if ( x < 0 )
    {
```

```

        x = 0 ;
        dx = 10 ;
        PlaySound ("chord.wav", NULL, SND_FILENAME | SND_ASYNC ) ;
    }
    else if ( x > ( r.right - wd ) )
    {
        x = r.right - wd ;
        dx = -10 ;
        PlaySound ("chord.wav", NULL, SND_FILENAME | SND_ASYNC ) ;
    }

    y += dy ;
    if ( y < 0 )
    {
        y = 0 ;
        dy = 10 ;
        PlaySound ("chord.wav", NULL, SND_FILENAME | SND_ASYNC ) ;
    }
    else if ( y > ( r.bottom - ht ) )
    {
        y = r.bottom - ht ;
        dy = -10 ;
        PlaySound ( "chord.wav", NULL, SND_FILENAME | SND_ASYNC );
    }

    BitBlt ( hdc, x, y, wd, ht, hmemdc, 0, 0, SRCCOPY ) ;
    ReleaseDC ( hWnd, hdc ) ;
}

```

From the **WndProc()** function you can observe that we have handled two new messages here—**WM_CREATE** and **WM_TIMER**. For these messages we have called the handlers **OnCreate()** and **OnTimer()** respectively. Let us now understand these handlers one by one

WM_CREATE and OnCreate()

The **WM_CREATE** message arrives whenever a new window is created. Since usually a window is created only once, the one-time activity that is to be carried out in a program is usually done in **OnCreate()** handler. In our program to make the ball move we need to display it at different places at different times. To do this it would be necessary to blit the ball image several times. However, we need to load the image only once. As this is a one-time activity it has been done in the handler function **OnCreate()**.

You are already familiar with the steps involved in preparing the image for blitting—loading the bitmap, creating a memory DC, making it compatible with screen DC and selecting the bitmap in the memory DC.

Apart from preparing the image for blitting we have also done some initialisations like setting up values in some variables to indicate the initial position of the ball. We have also called the **SetTimer()** function. This function tells Windows to post a message **WM_TIMER** into the message queue of our application every 50 milliseconds.

WM_TIMER and OnTimer()

If we are to perform an activity at regular intervals we have two choices:

- (a) Use a loop and monitor within the loop when is it time to perform that activity.
- (b) Use a Windows mechanism of timer. This mechanism when used posts a **WM_TIMER** message at regular intervals to our application.

The first method would seriously hamper the responsiveness of the program. If the control is within the loop and a new message arrives the message would not get processed unless the control goes out of the loop. The second choice is better because it makes the program event driven. That is, whenever **WM_TIMER** arrives

that time its handler does the job that we want to get executed periodically. At other times the application is free to handle other messages that come to its queue.

All that we have done in the **OnTimer()** handler is erase the ball from previous position and draw it at a new position. We have also checked if the ball has hit the boundaries of the window. If so we have played a sound file using the **PlaySound()** API function and then changed the direction of the ball.

A Few More Points...

A few more points worth noting before we close our discussion on animation...

- (a) One application can set up multiple timers to do different jobs at different intervals. Hence we need to pass the id of the timer that we want to set up to the **SetTimer()** function. In our case we have specified the id as 1.
- (b) For multiple timers Windows would post multiple **WM_TIMER** messages. Each time it would pass the timer id as additional information about the message.
- (c) For drawing as well as erasing the ball we have used the same function—**BitBlt()**. While erasing we have used the raster operation code **WHITENESS**. When we use this code the color values of the source pixels get ignored. Thus red colored pixels of ball would get ignored leading to erasure of the ball in the window.
- (d) The size of client area of the window can be obtained using the **GetClientRect()** API function.
- (e) We want that every time we run the application the initial position of the ball should be different. To ensure this we have generated its initial x, y coordinates using the standard library function **rand()**. However, this function doesn't

generate true random numbers. To ensure that we do get true random numbers, somehow we need to tie the random number generation with time, as time of each execution of our program would be different. This has been achieved by making the call

```
srand ( time ( NULL ) ) ;
```

Here **time()** is function that returns the time. We have further passed this time to the **srand()** function.

- (f) To be able to use **rand()** and **srand()** functions include the file 'stdlib.h'. Similarly for **time()** function to work include the file 'time.h'.
- (g) In the call to the **PlaySound()** function the first parameter is the name of the wave file that is to be played. If first parameter is filename then the second has to be NULL. The third parameter is a set of flags. **SND_FILENAME** indicates that the first parameter is the filename, whereas **SND_ASYNC** indicates that the sound should be played in the background.
- (h) To be able to use the **PlaySound()** function we need to link the library 'winmm.lib'. This is done by using 'Project | Settings' menu item. On selection of this item a dialog pops up. In the 'Link' tab of this dialog mention the name 'winmm.lib' in the 'Object / Library modules' edit box.
- (i) When the application terminates we have to instruct Windows not to send **WM_TIMER** messages to our application any more. For this we have called the **KillTimer()** API function passing to it the ID of the timer.

Windows, the Endless World...

The biggest hurdle in Windows programming is a sound understanding of its programming model. In this chapter and in the

last two I have tried to catch the essence of Windows' Event Driven Programming model. Once you have understood it thoroughly rest is just a matter of understanding and calling the suitable API functions to get your job done. Windows API is truly an endless world. It covers areas like Networking, Internet programming, Telephony, Drawing and Printing, Device I/O, Imaging, Messaging, Multimedia, Windowing, Database programming, Shell programming, to name a few. The programs that we have written have merely scratched the surface. No matter how many programs that we write under Windows, several still remain to be written. The intention of this chapter was to unveil before you, to give you the first glimpse of what is possible under Windows. The intention all along was not to catch fish for you but to show you how to catch fish so that you can do fishing all your life. Having made a sound beginning, rest is for you to explore. Good luck and happy fishing!

Summary

- (a) In DOS, programmers had to write separate graphics code for every new video adapter. In Windows, the code once written works on any video adapter.
- (b) A Windows program cannot draw directly on an output device like screen or printer. Instead, it draws to the logical display surface using device context.
- (c) When the window is displayed for the first time, or when it is moved or resized **OnPaint()** handler gets called.
- (d) It is necessary to obtain the device context before drawing text or graphics in the client area.
- (j) A device context is a structure containing information required to draw on a display surface. The information includes color of pen and brush, screen resolution, color palettes, etc.
- (e) To draw using a new pen or brush it is necessary to select them into the device context.

- (f) If we don't select any brush or pen into the device context then the drawing drawn in the client area would be drawn with the default pen (black pen) and default brush (white brush).
- (g) RGB is a macro representing the Red, Green and Blue elements of a color. RGB (0, 0, 0) gives black color, whereas, RGB (255, 255, 255) gives white color.
- (h) Animation involves repeatedly drawing the same image at successive positions.

Exercise

[A] State True or False:

- (a) Device independence means the same program is able to work using different screens, keyboards and printers without modifications to the program.
- (b) The WM_PAINT message is generated whenever the client area of the window needs to be redrawn.
- (c) The API function EndPaint() is used to release the DC.
- (d) The default pen in the DC is a solid pen of white color.
- (e) The pen thickness for the pen style other than PS_SOLID has to be 1 pixel.
- (f) BeginPaint() and GetDC() can be used interchangeably.
- (g) If we drag the mouse from (10, 10) to (110, 100), 100 WM_MOUSEMOVE messages would be posted into the message queue.
- (h) WM_PAINT message is raised when the window contents are scrolled.
- (i) With each DC a default monochrome bitmap of size 1 pixel x 1 pixel is associated.
- (j) The WM_CREATE message arrives whenever a window is displayed.

[B] Answer the following:

- (a) What is meant by Device Independent Drawing and how it is achieved?

- (b) Explain the significance of WM_PAINT message.
- (c) How Windows manages the code and various resources of a program?
- (d) Explain the Windows mechanism of timer.
- (e) What do you mean by capturing a mouse?
- (f) Write down the steps that need to be carried out to animate an object.

[C] Attempt the following:

- (a) Write a program, which displays "hello" at any place in the window where you click the left mouse button. If you click the right mouse button the color of subsequent hellos should change.
- (b) Write a program that would draw a line by joining the new point where you have clicked the left mouse button with the last point where you clicked the left mouse button.
- (c) Write a program to gradient fill the entire client area with shades of blue color.
- (d) Write a program to create chessboard like boxes (8 X 8) in the client area. If the window is resized the boxes should also get resized so that all the 64 boxes are visible at all times.
- (e) Write a program that displays only the upper half of a bitmap of size 40 x 40.
- (f) Write a program that displays different text in different colors and fonts at different places after every 10 seconds.

19 *Interaction With Hardware*

- Hardware Interaction
- Hardware Interaction, DOS Perspective
- Hardware Interaction, Windows Perspective
- Communication with Storage Devices
 - The *ReadSector()* Function
- Accessing Other Storage Devices
- Communication with Keyboard
 - Dynamic Linking
 - Windows Hooks
- Caps Locked, Permanently
- Did You Press It TTwwiiccee....
- Mangling Keys
- KeyLogger
- Where is This Leading
- Summary
- Exercise

There are two types of Windows programmers those who are happy in knowing the things the way they are under Windows and those who wish to know why the things are the way they are. This chapter is for the second breed of programmers. They are the real power users of Windows. Because it is they who first understand the default working of different mechanisms that Windows uses and then are able to make those mechanisms work to their advantage. The focus here would be restricted to mechanisms that are involved in interaction with the hardware under the Windows world. Read on and I am sure you would be on your path to become a powerful Windows programmer.

Hardware Interaction

Primarily interaction with hardware suggests interaction with peripheral devices. However, its reach is not limited to interaction with peripherals. The interaction may also involve communicating with chips present on the motherboard. Thus more correctly, interaction with hardware would mean interaction with any chip other than the microprocessor. During this interaction one or more of the following activities may be performed:

- (a) Reacting to events that occur because of user's interaction with the hardware. For example, if the user presses a key or clicks the mouse button then our program may do something.
- (b) Reacting to events that do not need explicit user's interaction. For example, on ticking of a timer our program may want to do something.
- (c) Explicit communication from a program without the occurrence of an event. For example, a program may want to send a character to the printer, or a program may want to read/write the contents of a sector from the hard disk.

Let us now see how this interaction is done under different platforms.

Hardware Interaction, DOS Perspective

Under DOS whenever an external event (like pressing a key or ticking of timer) occurs a signal called hardware interrupt gets generated. For different events there are different interrupts. As a reaction to the occurrence of an interrupt a table called Interrupt Vector Table (IVT) is looked up. IVT is present in memory. It is populated with addresses of different BIOS routines during booting. Depending upon which interrupt has occurred the Microprocessor picks the address of the appropriate BIOS routine from IVT and transfers execution control to it. Once the control reaches the BIOS routine, the code in the BIOS routine interacts with the hardware. Naturally, for different interrupts different BIOS routines are called. Since these routines serve the interrupts they are often called ‘Interrupt Service Routines’ or simply ISRs.

Refer Figure 19.1 to understand this mechanism.

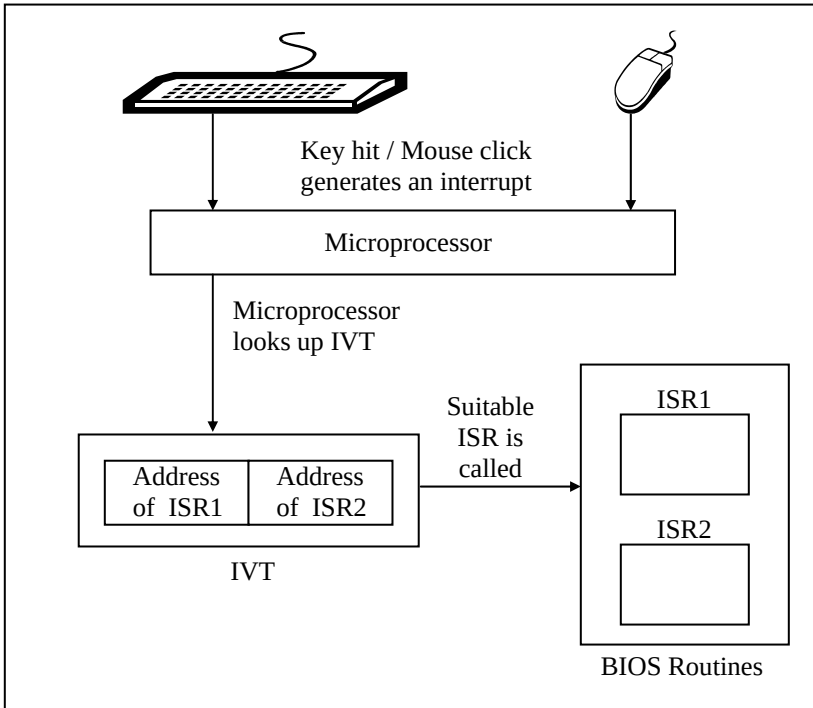


Figure 19.1

If we want that instead of the default ISR our routine should get called then it is necessary to store the address of this routine in IVT. Once this is done whenever a hardware interrupt occurs our routine's address from IVT is picked up and the control is transferred to our routine. For example, we may register our ISR in IVT to gain finer control over the way key-hits from the keyboard are tackled. This finer control may involve changing codes of keys or handling hitting of multiple keys simultaneously.

Explicit communication with the hardware can be done in four different ways. These are shown in Figure 19.2.

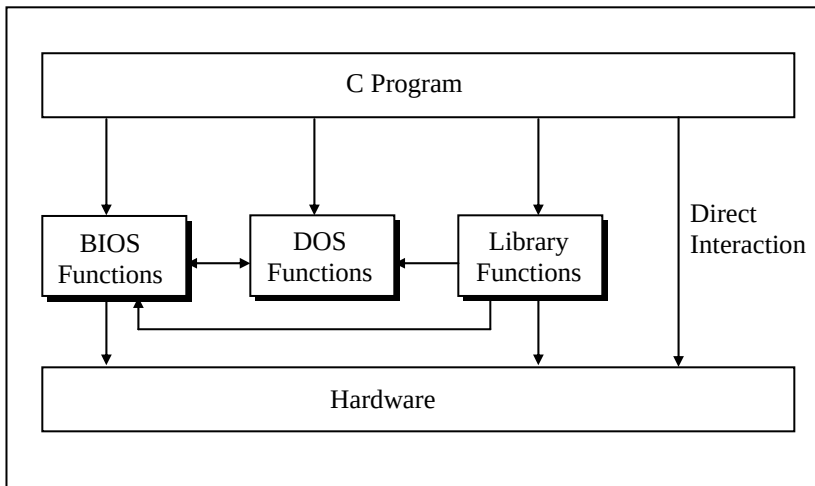


Figure 19.2

Let us now discuss the pros and cons of using these different methods to interact with the hardware.

(a) Calling DOS Functions

To interact with the hardware a program can call DOS functions. These functions can either directly interact with the hardware or they may call BIOS functions which in turn interact with the hardware. As a result, the programmer is not required to know all the hardware details to be able to interact with it. However, since DOS functions do not have names they have to be called through the mechanism of interrupts. This is difficult since the programmer has to remember interrupt service numbers for calling different DOS functions. Moreover, communication with these functions has to be done using CPU registers. This leads to lot of difficulties since different functions use different registers for communication. So one has to know details of different CPU registers, how to use them, which one to use when, etc.

(b) Calling BIOS Functions

DOS functions can carry out jobs like console I/O, file I/O, printing, etc. For other operations like generating graphics, carrying out serial communication, etc. the program has to call another set of functions called ROM-BIOS functions. Note that there are some functions in ROM-BIOS that do same jobs as equivalent DOS functions. BIOS functions suffer from the same difficulty as DOS functions—they do not have names. Hence they have to be called using interrupts and involve heavy usage of registers.

(c) Calling Library Functions

We can call library functions which in turn can call DOS/BIOS functions to carry out the interaction with hardware. Good examples of these functions are **printf()** / **scanf()** / **getch()** for interaction with console, **absread()** / **abswrite()** for interaction with disk, **bioscom()** for interaction with serial port, etc. But the library doesn't have a parallel function for every DOS/BIOS function. Hence at some point of time one has to learn how to call DOS/BIOS functions.

(d) Directly interacting with the hardware

At times the programs are needed to directly interact with the hardware. This has to be done because either there are no library functions or DOS/BIOS functions to do this, or if they are there their reach is limited. For example, while writing good video games one is required to watch the status of multiple keys simultaneously. The library functions as well as the DOS/BIOS functions are unable to do this. At such times we have to interact with the keyboard controller chip directly.

However, direct interaction with the hardware is difficult because one has to have good knowledge of technical details of the chip to be able to do so. Moreover, not every technical detail about how the hardware from a particular manufacturer works is well documented.

Hardware Interaction, Windows Perspective

Like DOS, under Windows too a hardware interrupt gets generated whenever an external event occurs. As a reaction to this signal a table called Interrupt Descriptor Table (IDT) is looked up and a corresponding routine for the interrupt gets called. Unlike DOS the IDT contains addresses of various kernel routines (instead of BIOS routines). These routines are part of the Windows OS itself. When the kernel routine is called, it in turn calls the ISR present in the appropriate device driver. This ISR interacts with the hardware. Two questions may now occur to you:

- (a) Why the kernel routine does not interact with the hardware directly?
- (b) Why the ISR of the device driver not registered directly in the IDT?

Let us find answer to the first question. Every piece of hardware works differently than the other. As new pieces of hardware come into existence new code has to be written to be able to interact with them. If this code is written in the kernel then the kernel would have to be rewritten and recompiled every time a new hardware comes into existence. This is practically impossible. Hence the new code to interact with the device is written in a separate program called device driver. With every new piece of hardware a new device driver is provided. This device driver is an extension of the OS itself.

Let us now answer the second question. Out of the several components of Windows OS a component called kernel is tightly integrated with the processor architecture. If the processor architecture changes then the kernel is bound to change. One of goals of Windows NT family was to keep the other components of OS and the device drivers portable across different microprocessor architectures. All processor architectures may not use IDT for the registration and lookup mechanism. So, had registration of the device driver's ISR in IDT been allowed, then the mechanism

would fail on processors which do not use IDT, thereby compromising portability of device drivers.

Refer Figure 19.3 for understanding the interrupt handling mechanism under Windows.

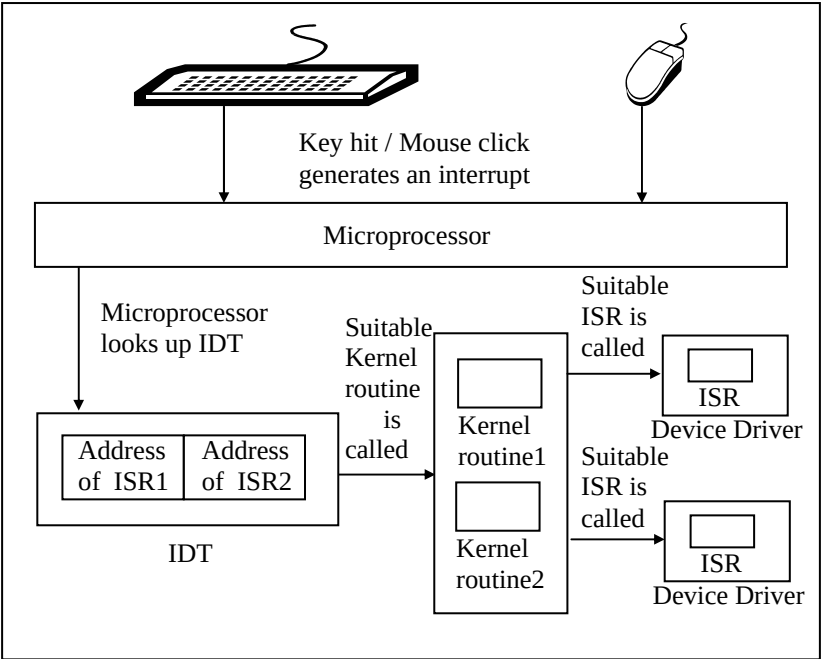


Figure 19.3

If we are to gain finer control while reacting to interrupts we would be required to write a device driver containing a new ISR to do so.

Under Windows explicit communication with hardware is much different than the way it was done under DOS. This is primarily because under Windows every device is shared amongst multiple applications running in memory. To avoid conflict between different programs accessing the same device simultaneously

Windows does not permit an application program to directly access any of the devices. Instead it provides several API functions to carry out the interaction. These functions have names so calling them is much easier than calling DOS/BIOS functions. When we call an API function to interact with a device, it in turn accesses the device driver program for the device. It is the device driver program that finally accesses the device. There is a standard way in which an application can communicate with the device driver. It is device driver's responsibility to ensure that multiple requests coming from different applications are handled without causing any conflict. In the sections to follow we would see how to communicate with the device driver to be able to interact with the hardware.

One last question—won't the API change if a new device comes into existence? No it won't. That is the beauty of the Windows architecture. All that would change is the device driver program for the new device. The API functions that we would need to interact with this new device driver would remain same. This is shown in Figure 19.4

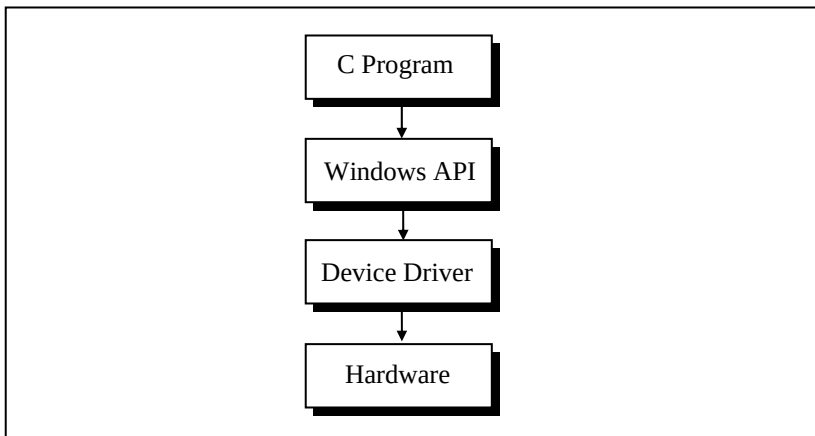


Figure 19.4

Communication with Storage Devices

Since DOS is commercially dead the rest of the chapter would focus on communication with the devices under Windows platform. We would illustrate this with the help of several programs.

Let us begin with the one that interacts with the simplest storage device, namely the floppy disk. Rather than the physical structure of the floppy disk it is the way the stored information is laid out and managed that concerns programmers most. Let us understand how the information is laid out on a floppy disk. Each floppy disk consists of four logical parts—Boot Sector, File Allocation Table (FAT), Directory and Data space. Of these, the Boot Sector contains information about how the disk is organized. That is, how many sides does it contain, how many tracks are there on each side, how many sectors are there per track, how many bytes are there per sector, etc. The files and the directories are stored in the Data Space. The Directory contains information about the files like its attributes, name, size, etc. The FAT contains information about where the files and directories are stored in the data space. Figure 19.5 shows the four logical parts of a 1.44 MB disk.